

# Byte Alignment & Ordering

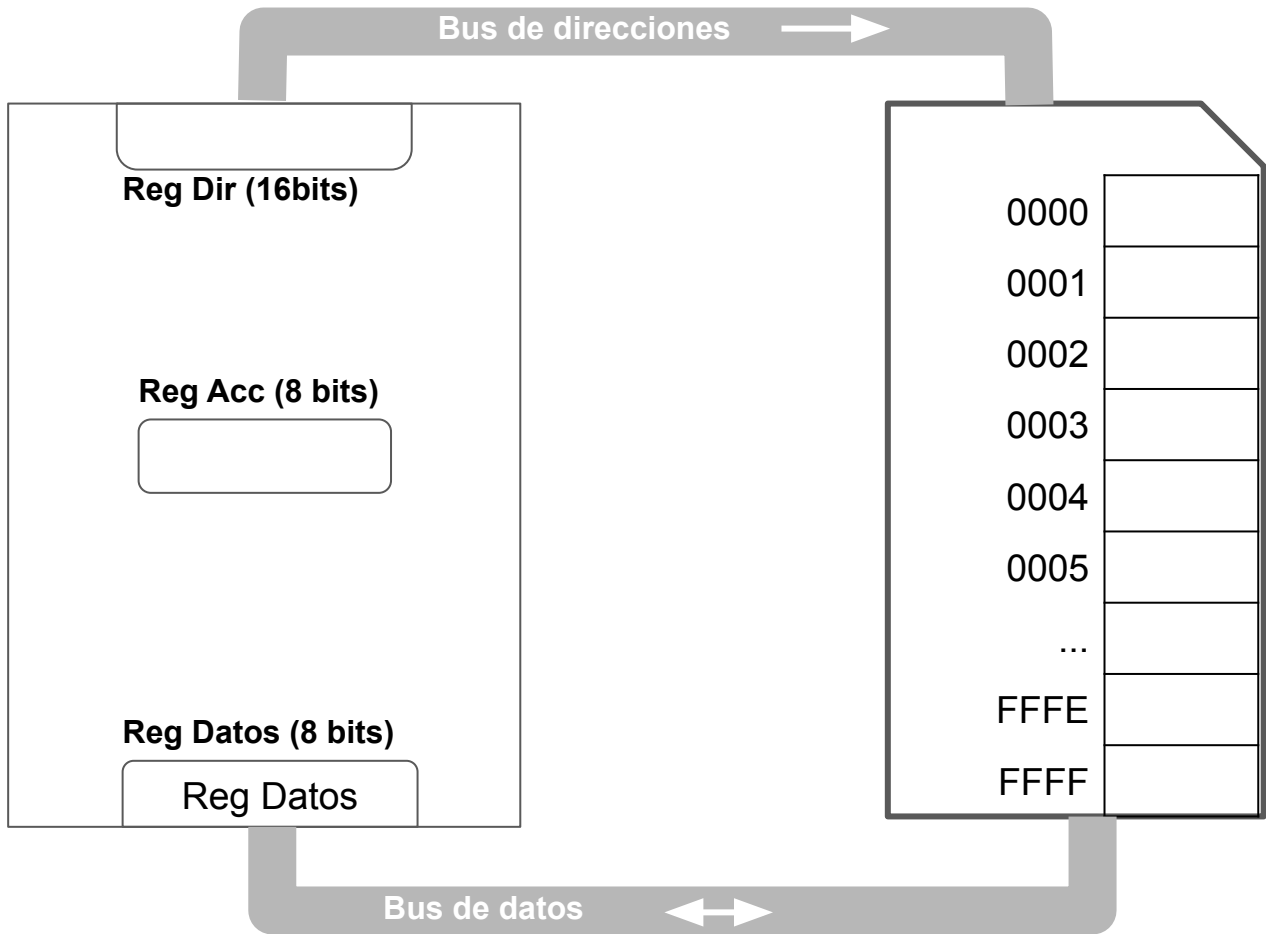
Unidad 0.1.5

## **Palabra (word)**

Cuando nos referimos a una palabra (word) en una computadora estamos hablando de una unidad de medida de información. Esta unidad es fija. Cada procesador define esta unidad en bits. Ejemplos:

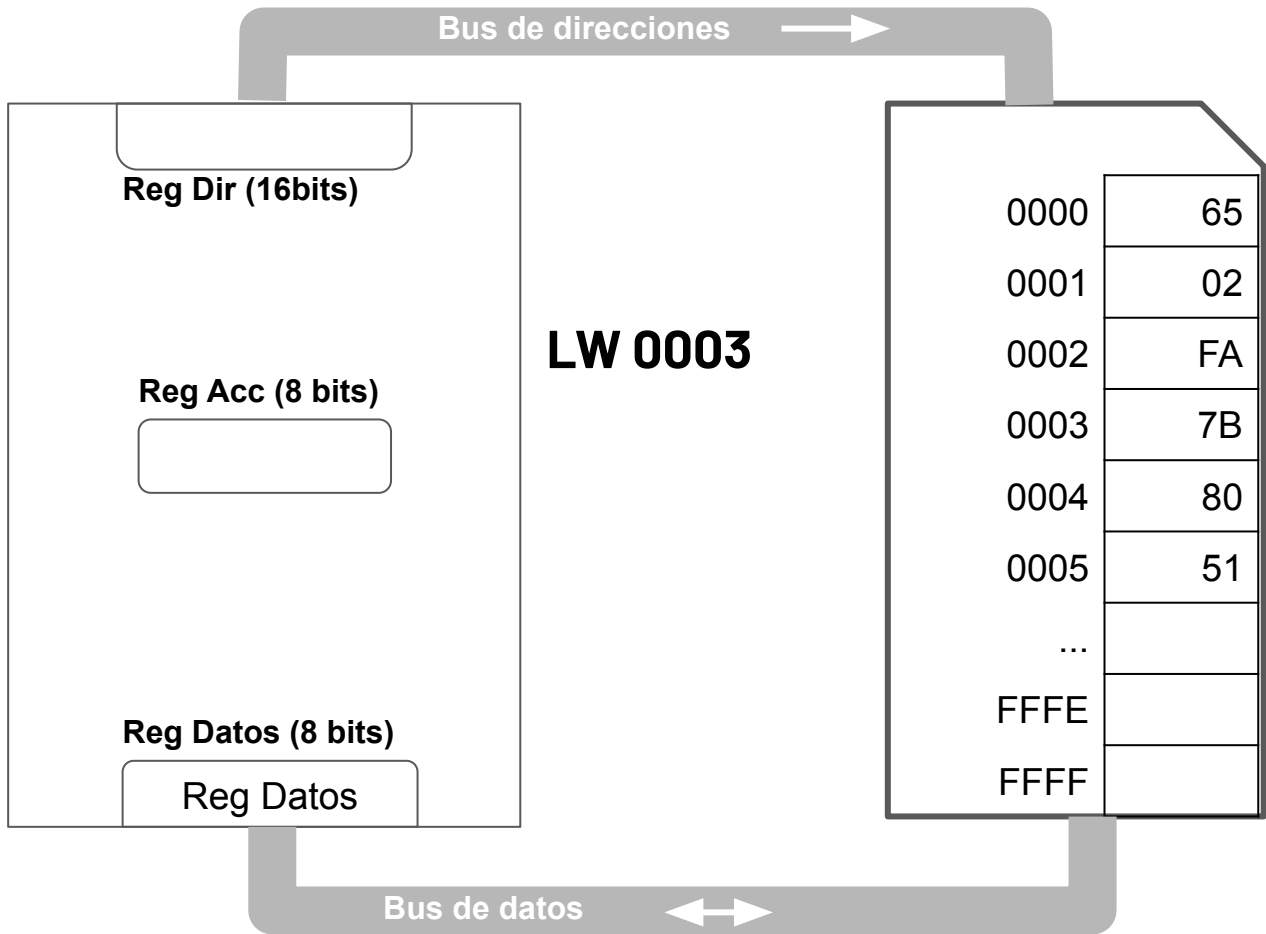
- MC14500B (CPU de 1 bit)
- Intel 4004 (CPU de 4 bits)
- MOS6502 , Zilog Z80, Intel 8008 , Intel 8080 (CPUs de 8 bits)
- Motorola 68k , Intel 8086, Intel 80286 (CPUs de 16 bits)
- Intel 80386, MIPS... (CPUs de 32 bits)
- AMD64 Opteron, Intel x86\_64... (CPUs de 64 bits)

# Arquitectura de 8 bits



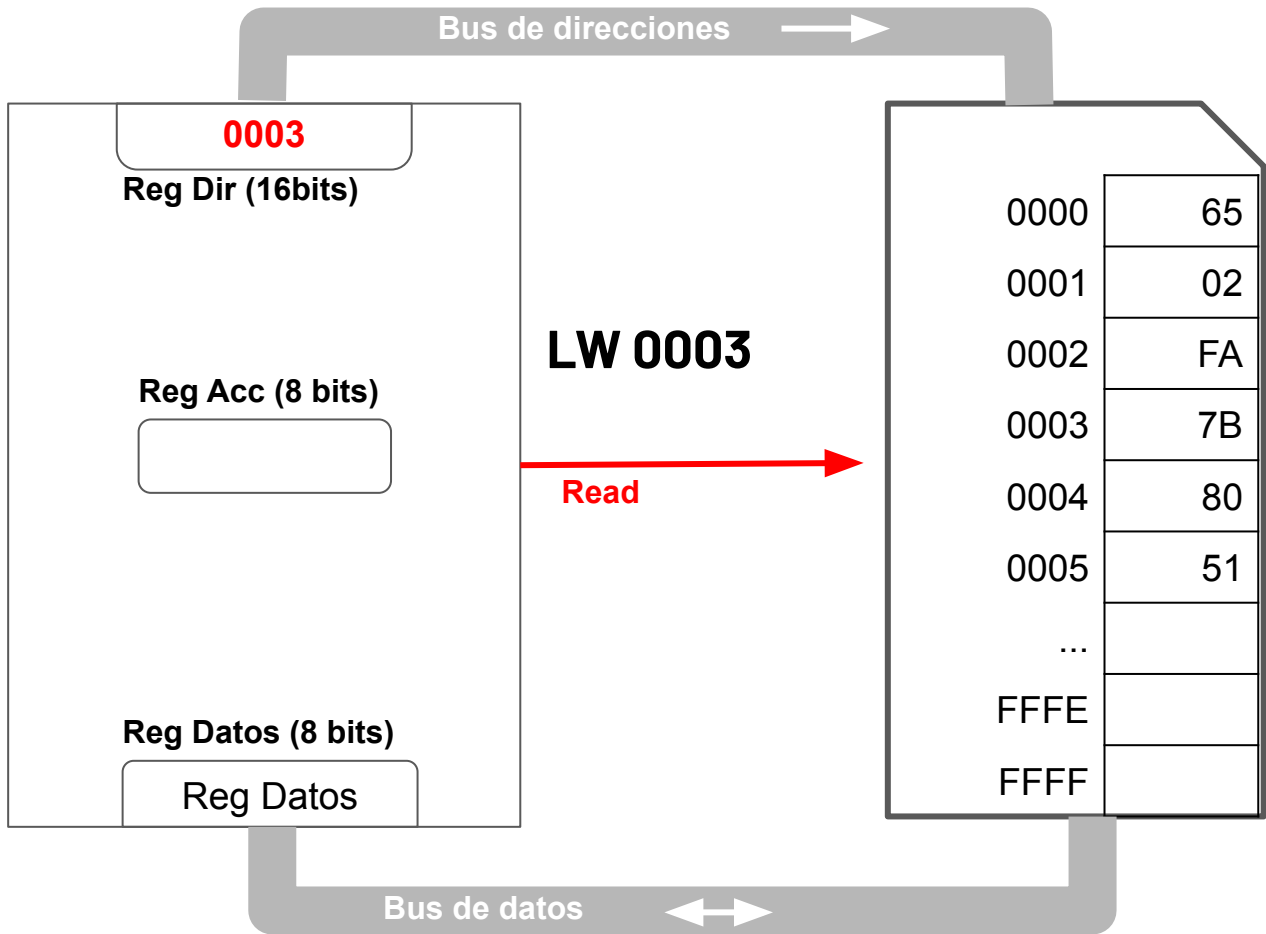
Existen 64KB máximos de memoria ( $2^{16}$ ). Cada posición de memoria almacena un byte. El registro acumulador es de un byte (word). Usamos LW dir para acceder a un dato.

# Arquitectura de 8 bits



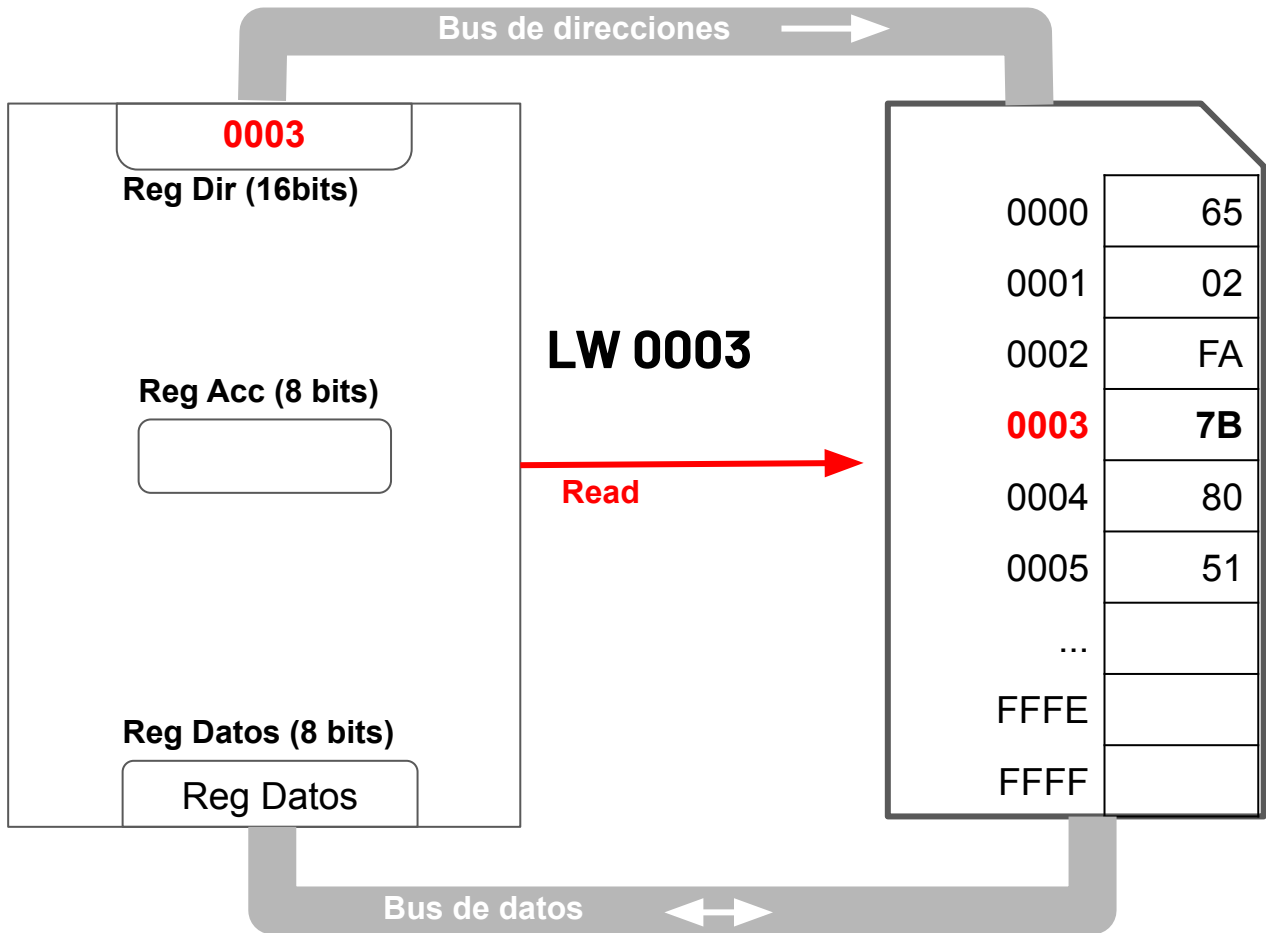
Existen 64KB máximos de memoria. Cada posición de memoria almacena un byte. El registro acumulador es de un byte (word). Usamos LW dir para acceder a un dato.

# Arquitectura de 8 bits



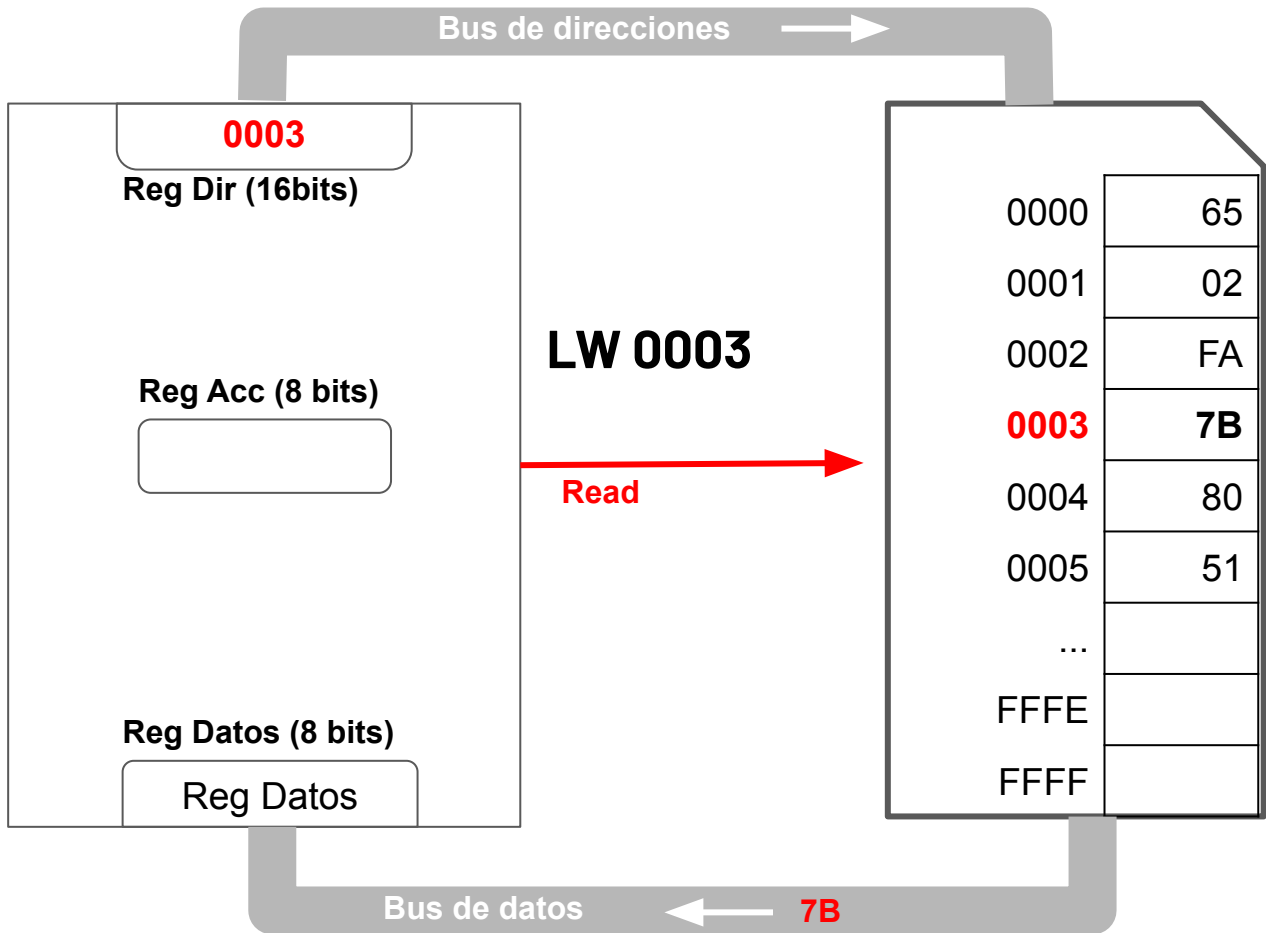
Existen 64KB máximos de memoria. Cada posición de memoria almacena un byte. El registro acumulador es de un byte (word). Usamos LW dir para acceder a un dato.

# Arquitectura de 8 bits



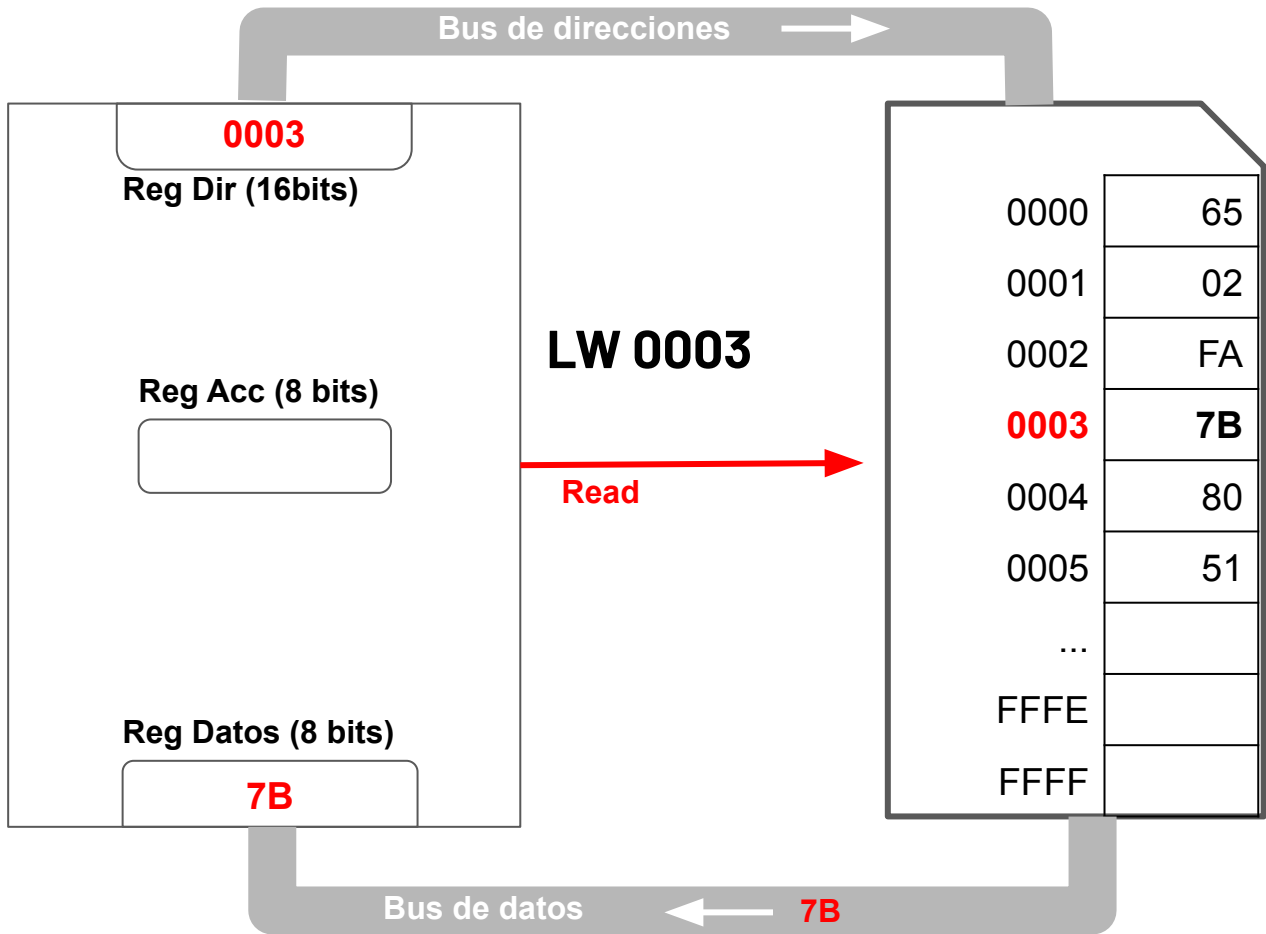
Existen 64KB máximos de memoria. Cada posición de memoria almacena un byte. El registro acumulador es de un byte (word). Usamos LW dir para acceder a un dato.

# Arquitectura de 8 bits



Existen 64KB máximos de memoria. Cada posición de memoria almacena un byte. El registro acumulador es de un byte (word). Usamos LW dir para acceder a un dato.

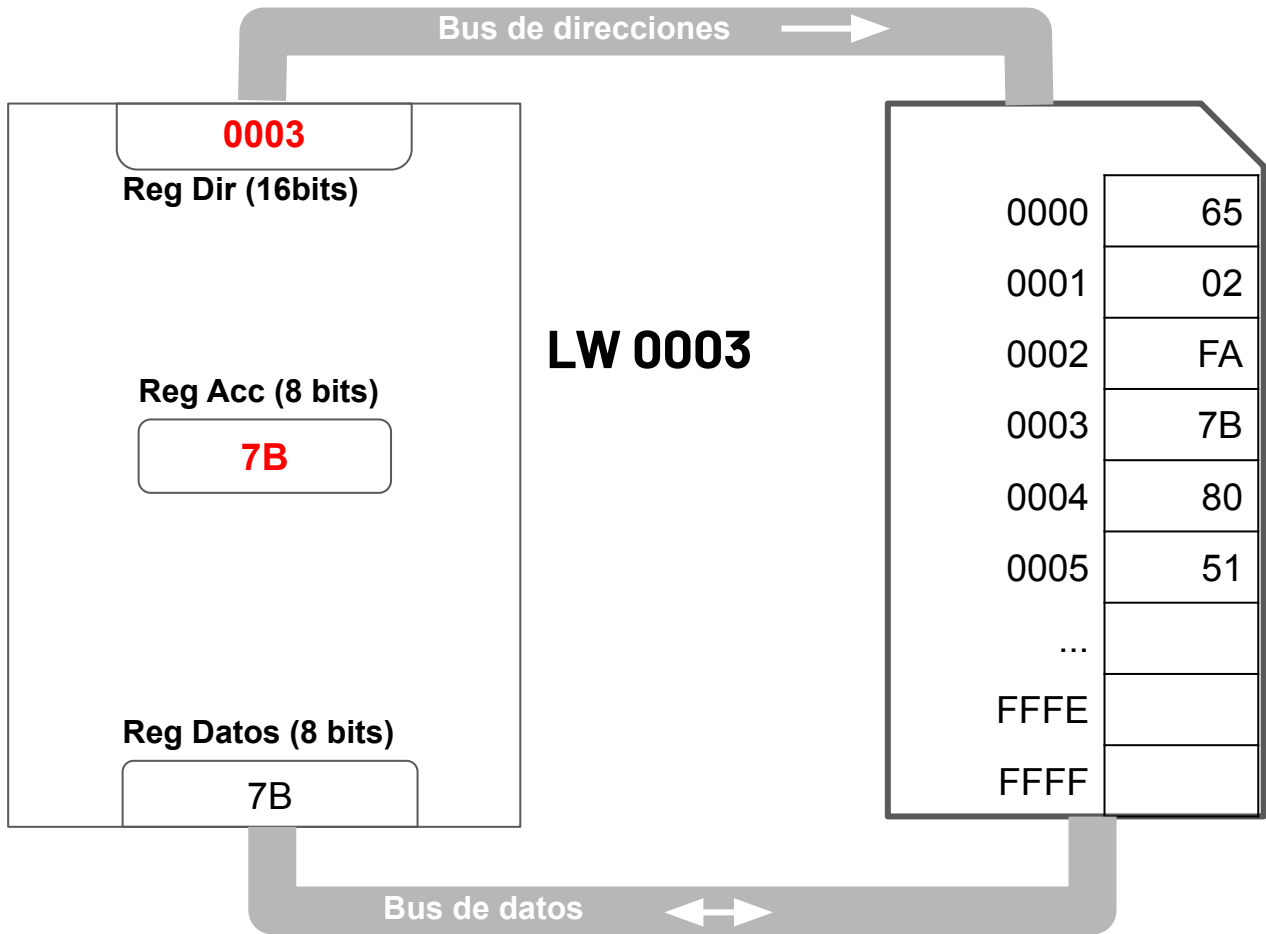
# Arquitectura de 8 bits



Existen 64KB máximos de memoria. Cada posición de memoria almacena un byte. El registro acumulador es de un byte (word). Usamos LW dir para acceder a un dato.

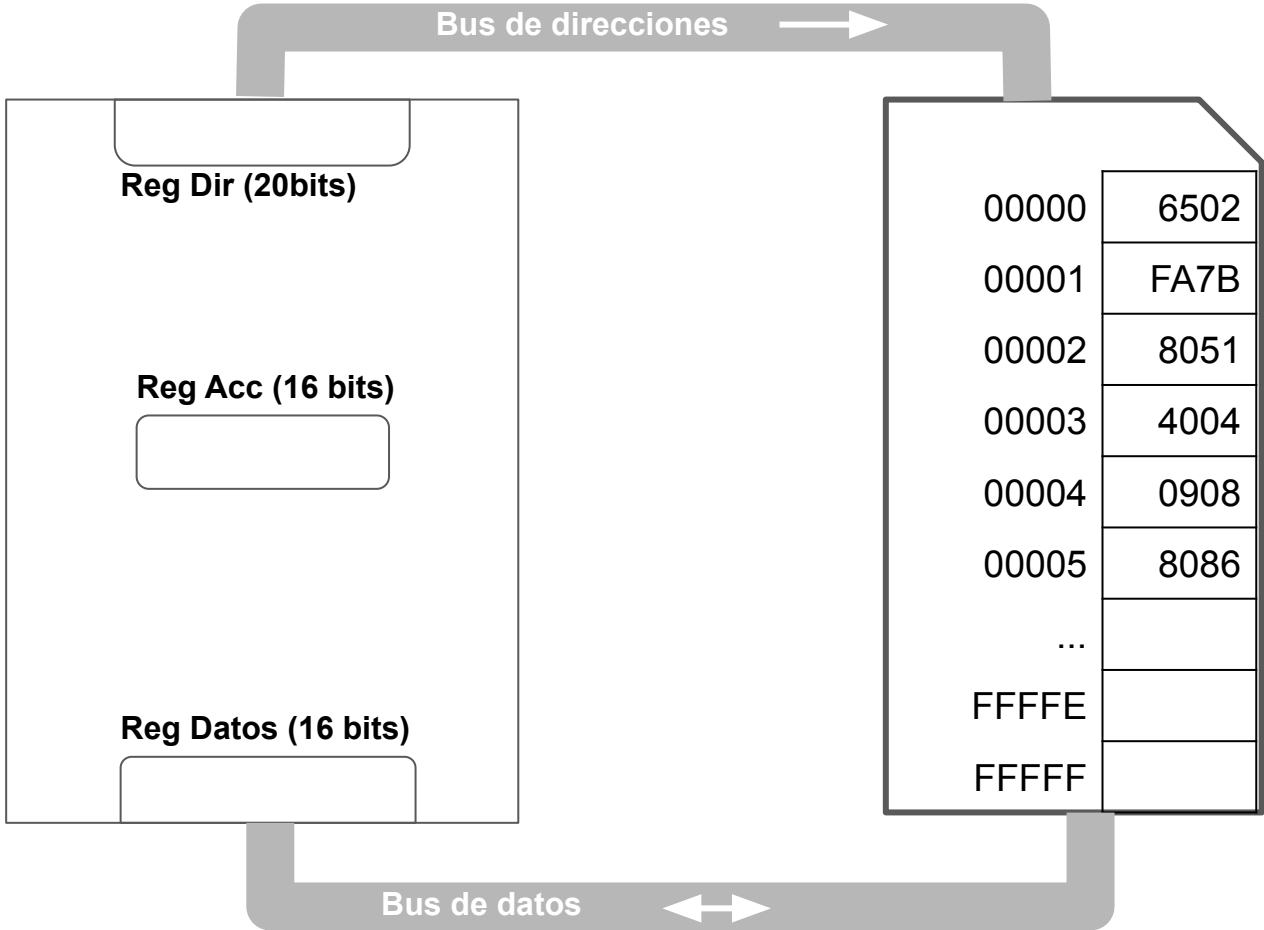


# Arquitectura de 8 bits



Existen 64KB máximos de memoria. Cada posición de memoria almacena un byte. El registro acumulador es de un byte (word). Usamos LW dir para acceder a un dato.

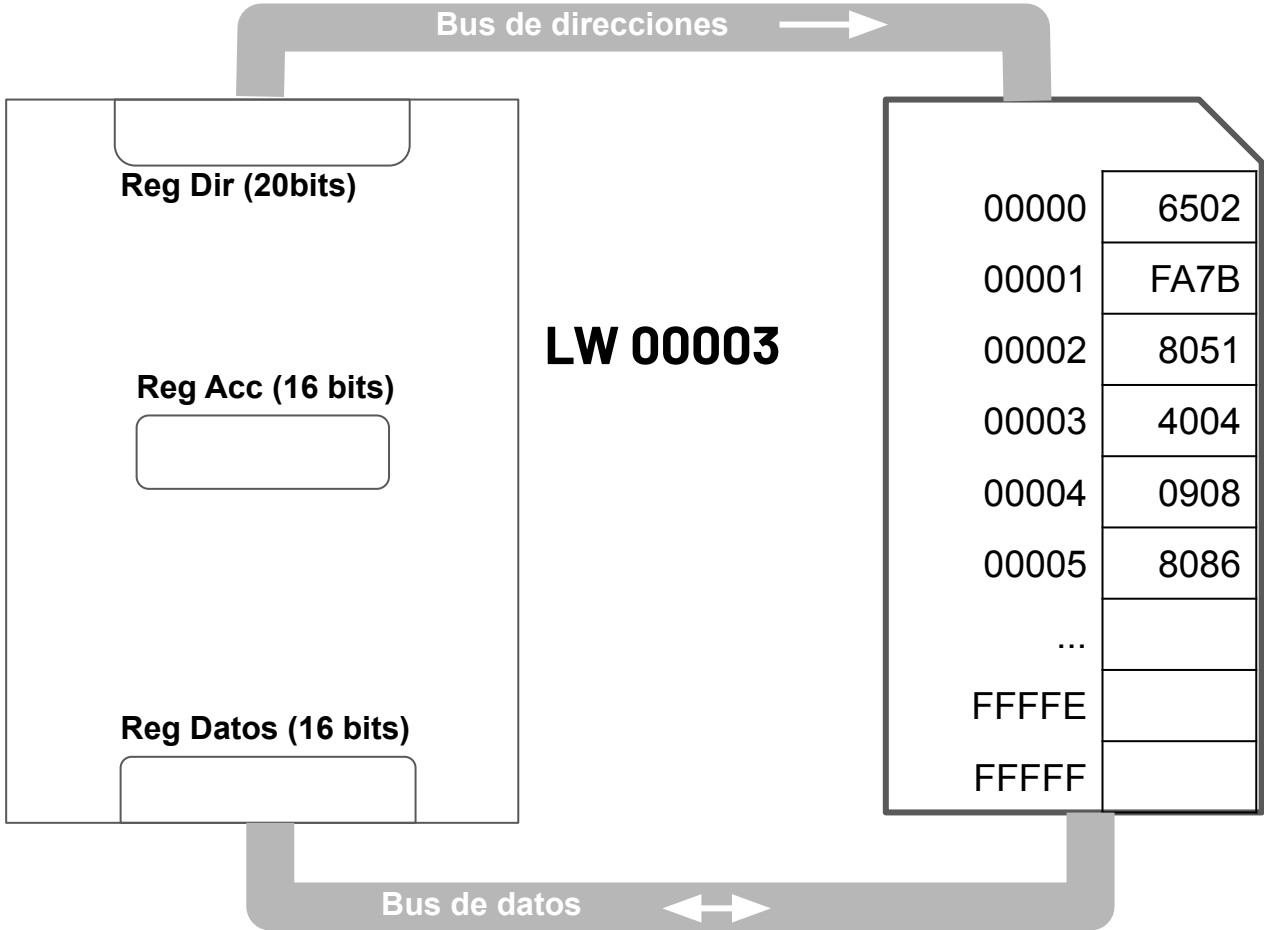
# Arquitectura de 16 bits (dir a la palabra)



Usamos 20 bits para direcciones, cada posición de memoria de 2 bytes, 2MB ( $2^{21}$ ) totales o 1M words ( $2^{20}$ ). La memoria usa 20 bits de dirección.

Para leer un word usamos LW y la dirección.

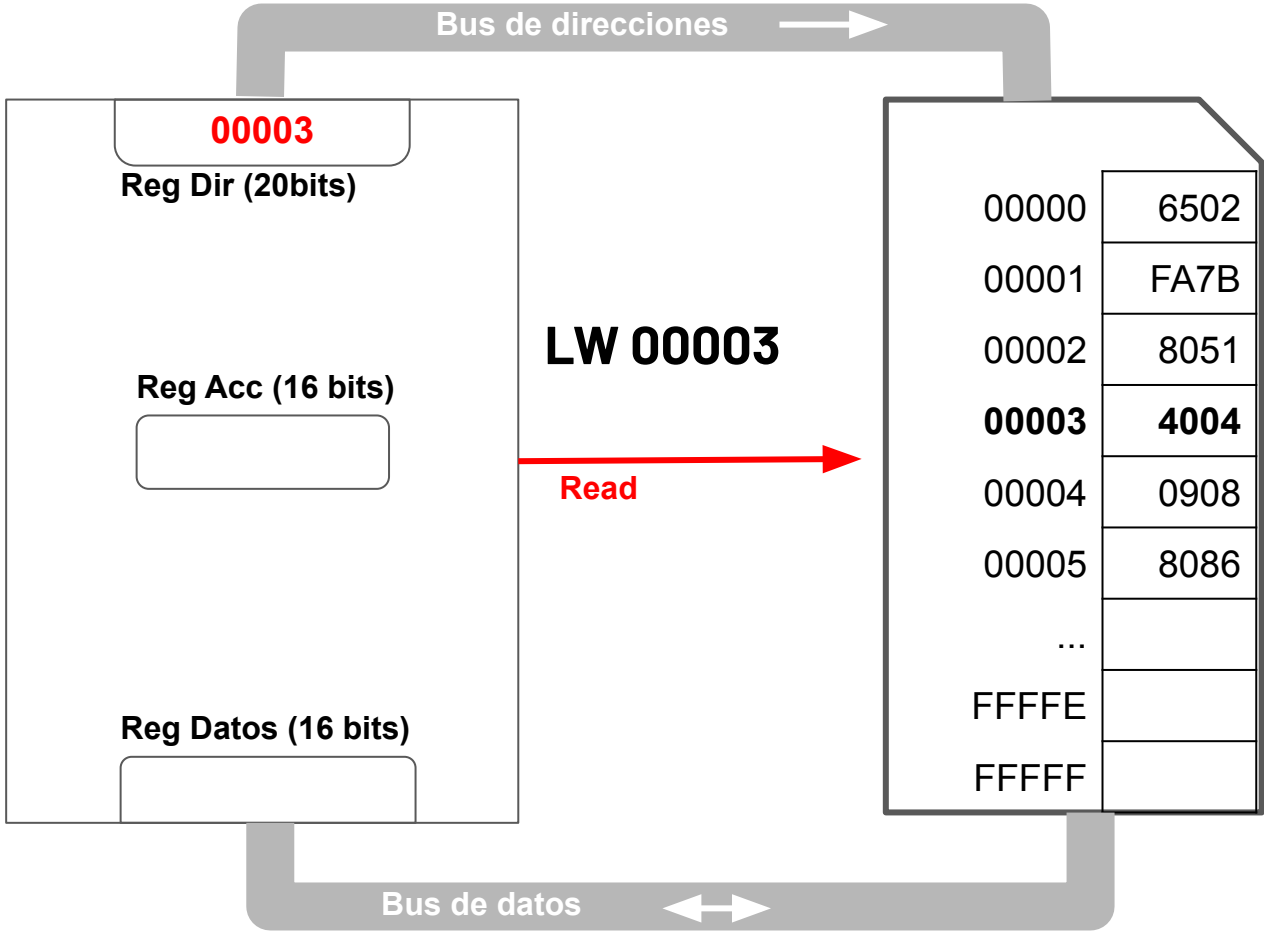
# Arquitectura de 16 bits (dir a la palabra)



Usamos 20 bits para direcciones, cada posición de memoria de 2 bytes, 2MB totales o 1M words.

Para leer un word usamos LW y la dirección.

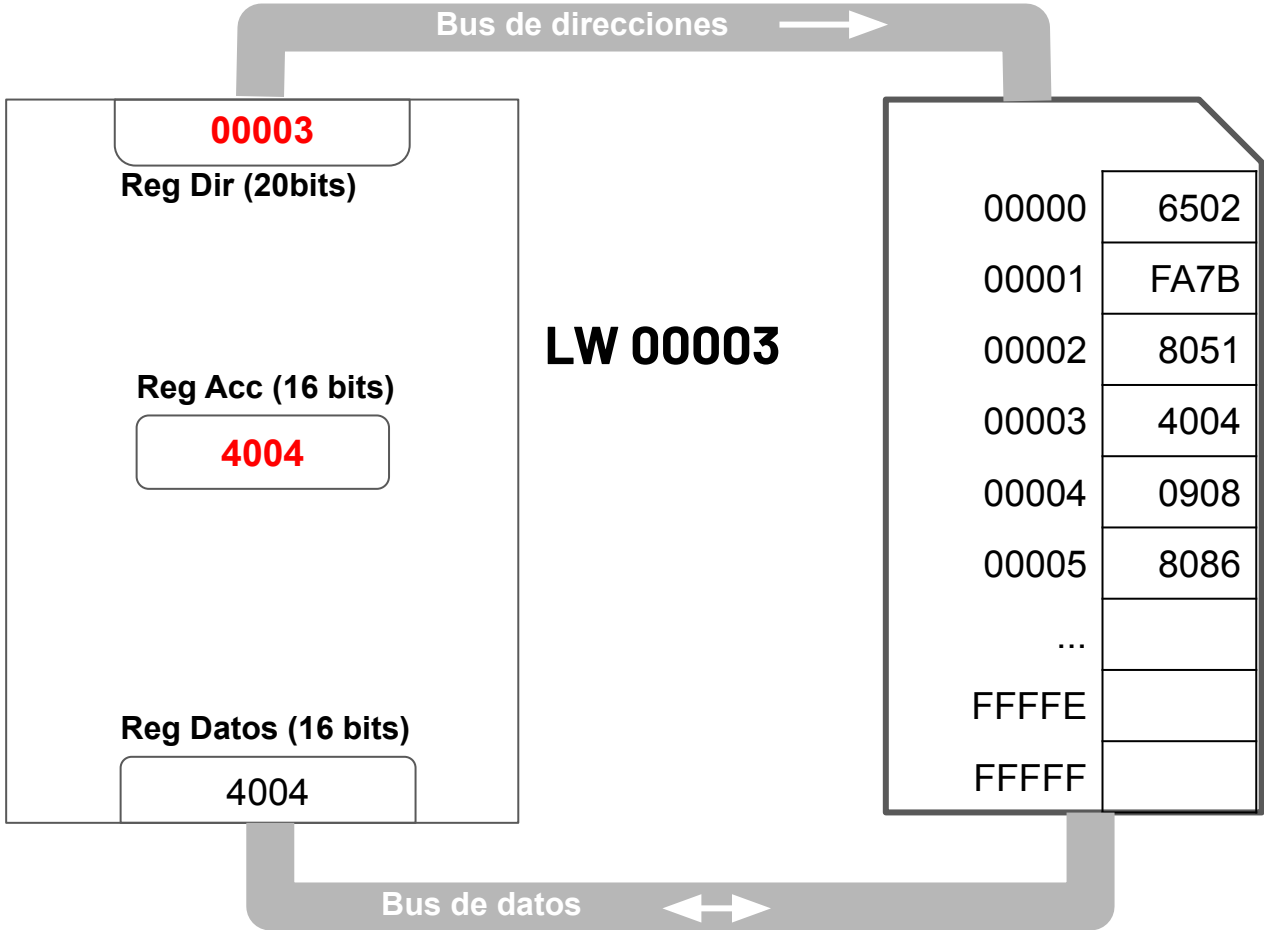
# Arquitectura de 16 bits (dir a la palabra)



Usamos 20 bits para direcciones, cada posición de memoria de 2 bytes, 2MB totales o 1M words.

Para leer un word usamos LW y la dirección.

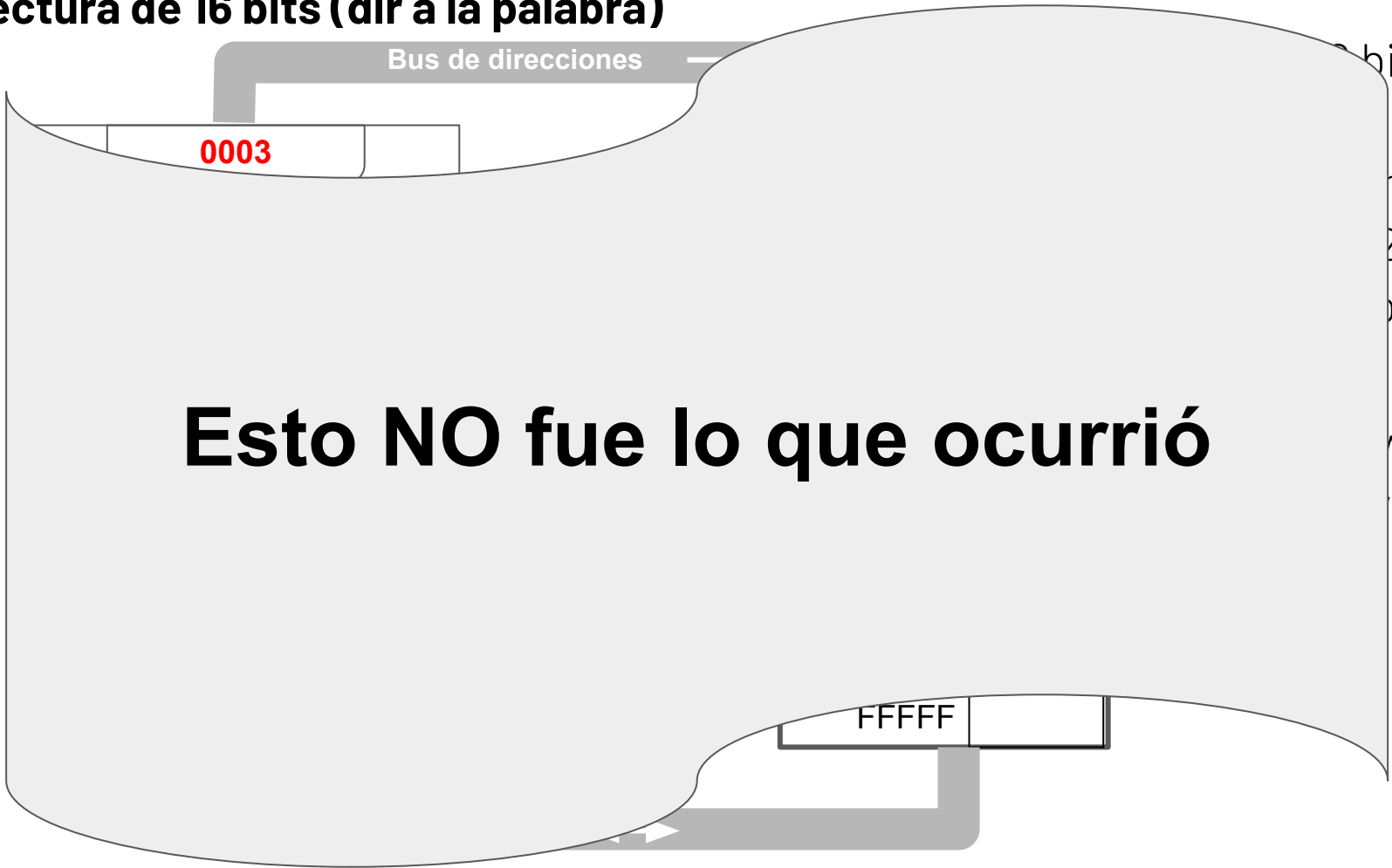
# Arquitectura de 16 bits (dir a la palabra)



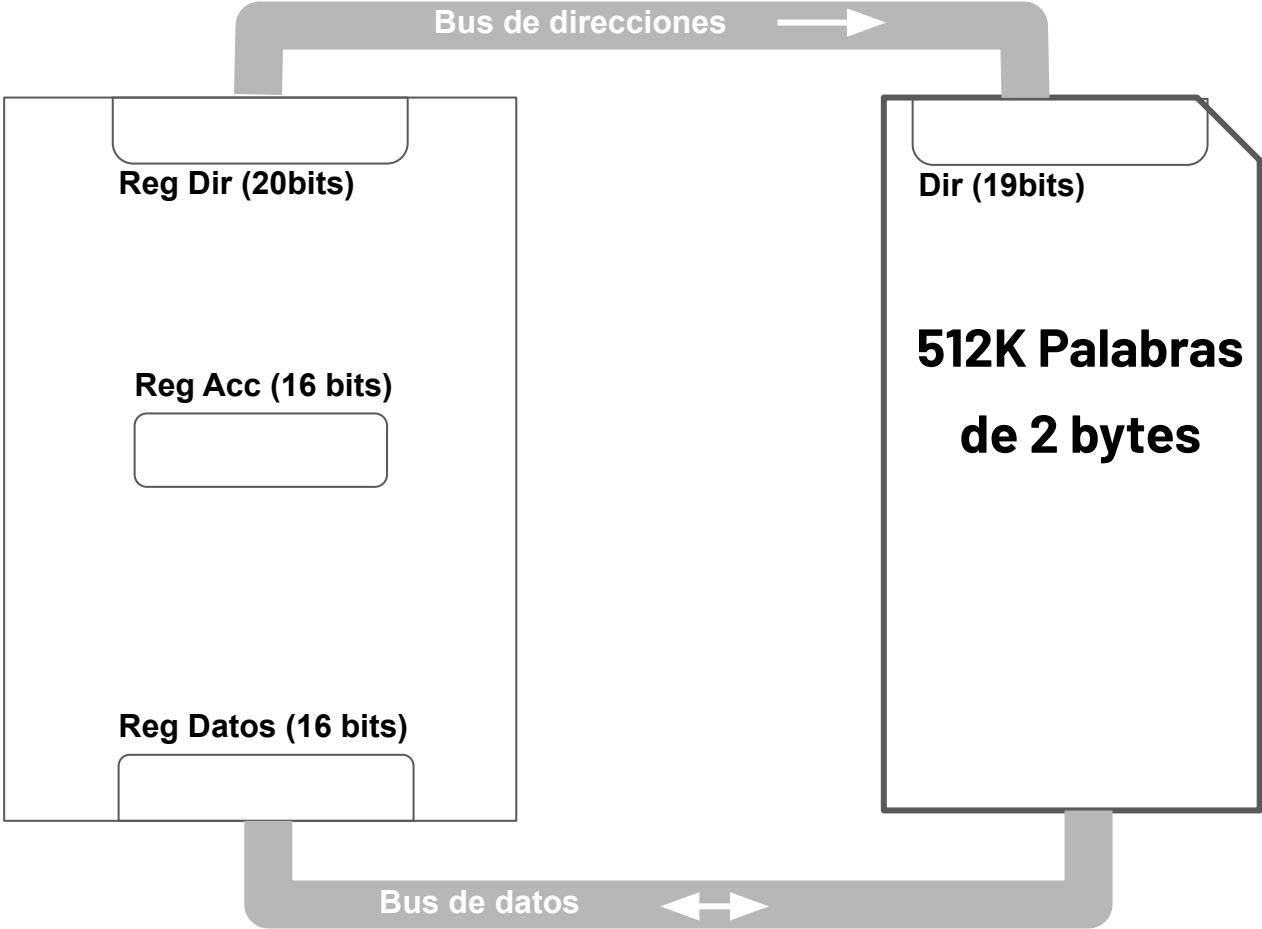
Usamos 20 bits para direcciones, cada posición de memoria de 2 bytes, 2MB totales o 1M words.

Para leer un word usamos LW y la dirección.

# Arquitectura de 16 bits (dir a la palabra)

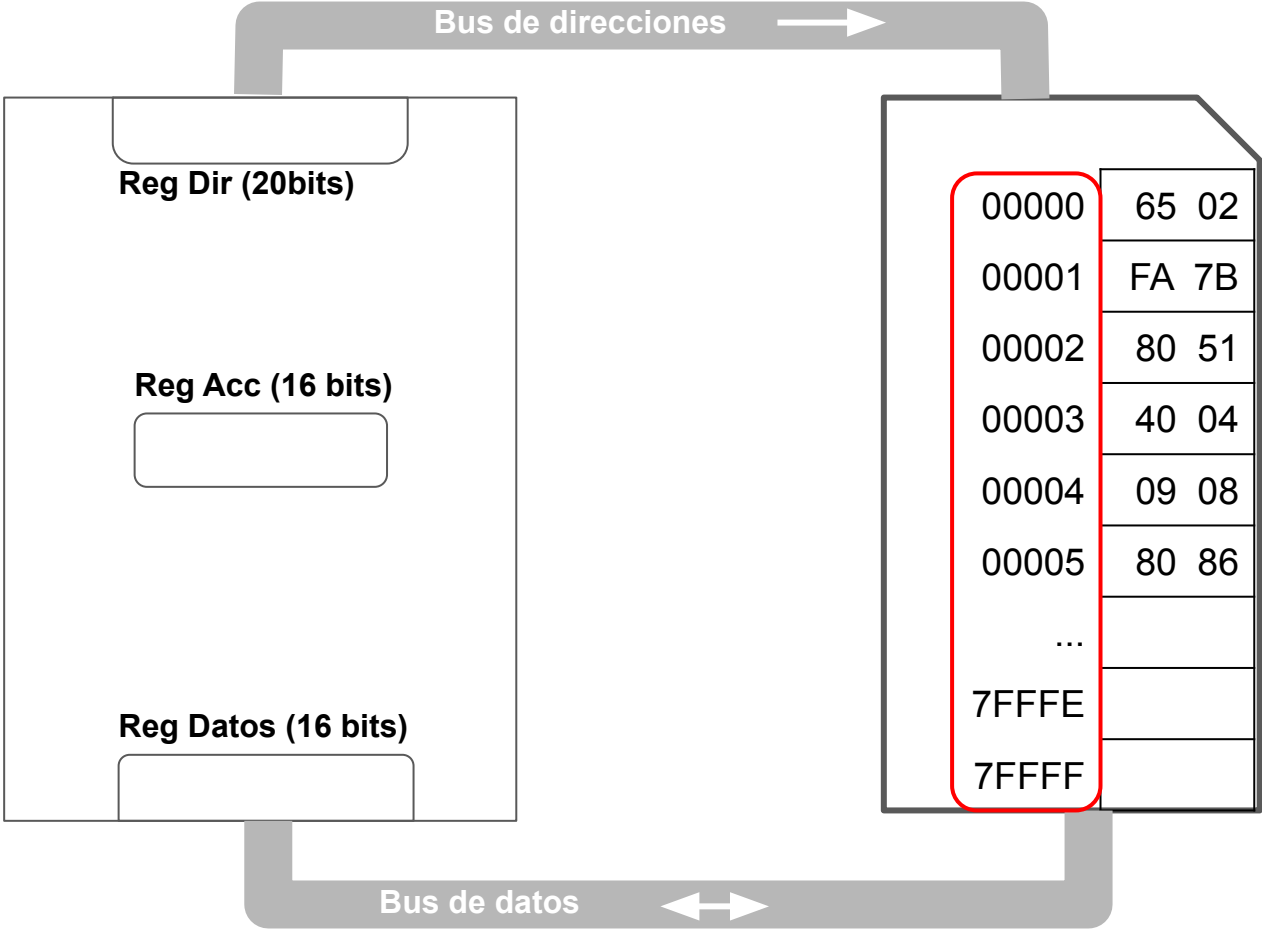


# Arquitectura de 16 bits (dir al byte)



Si desde la CPU se direcciona al byte, con 20 bits, se direccionan entonces  $2^{20}$  bytes, o sea 1MB. Pero cada palabra de memoria tiene 2 bytes. Por ende hay  $2^{19}$  posiciones de 2 bytes ( $512K \times 2$ )

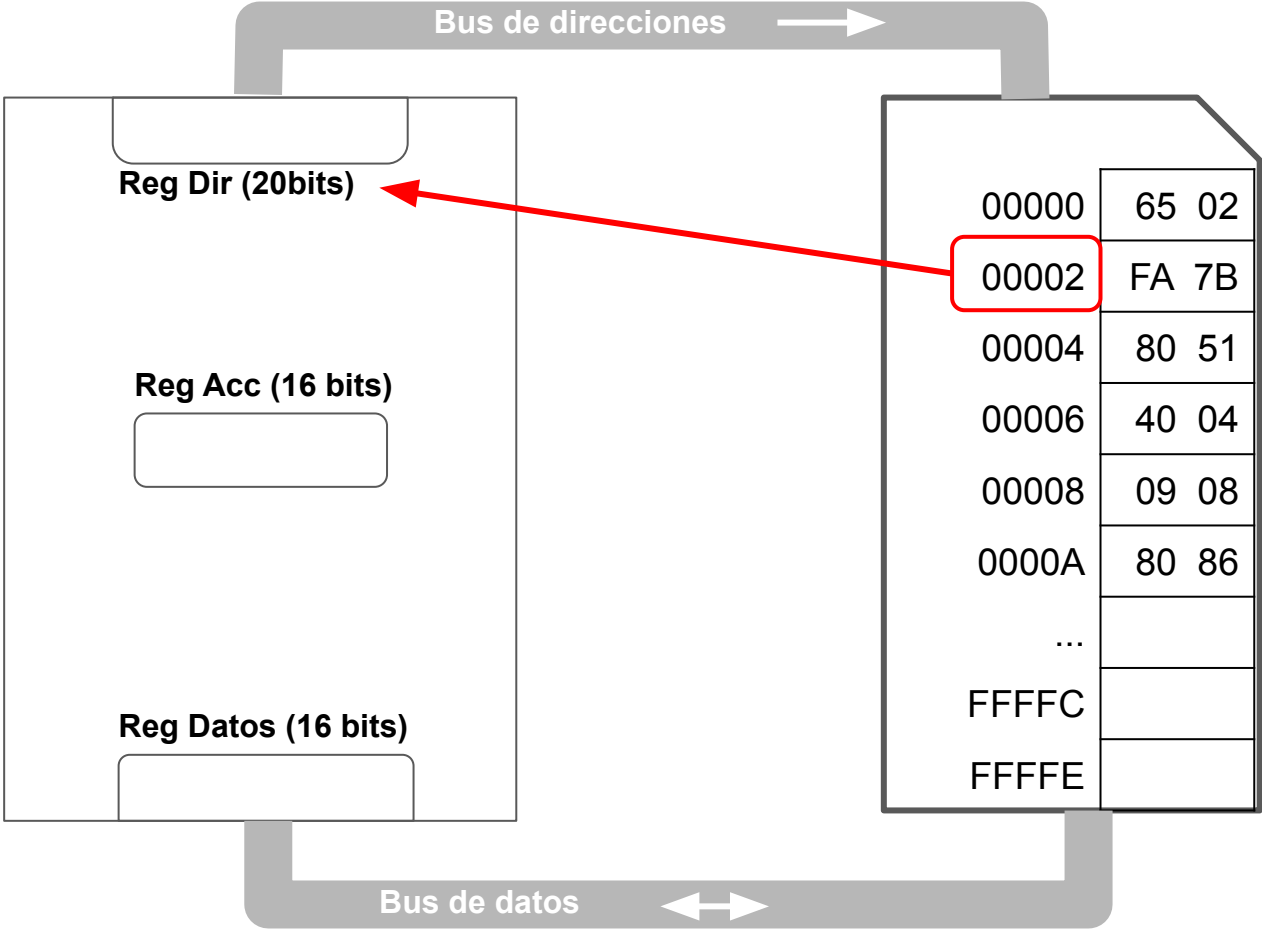
# Arquitectura de 16 bits (dir al byte)



Vemos que cada palabra tiene direcciones de 19 bits (00000 a 7FFFF). Pero la CPU direcciona al byte, entonces no usamos la dirección del word de memoria sino la dirección del byte de la CPU...

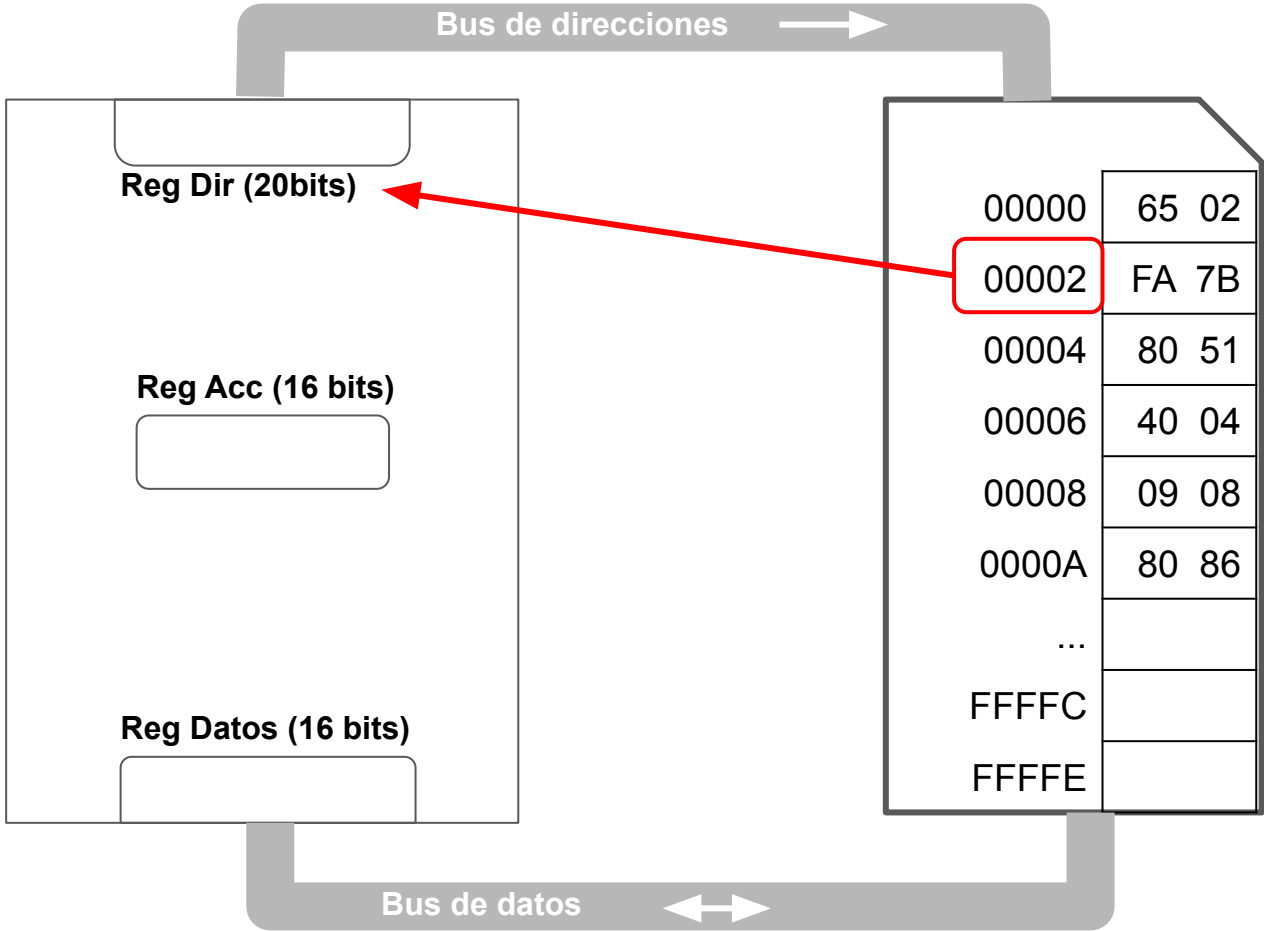


# Arquitectura de 16 bits (dir al byte)



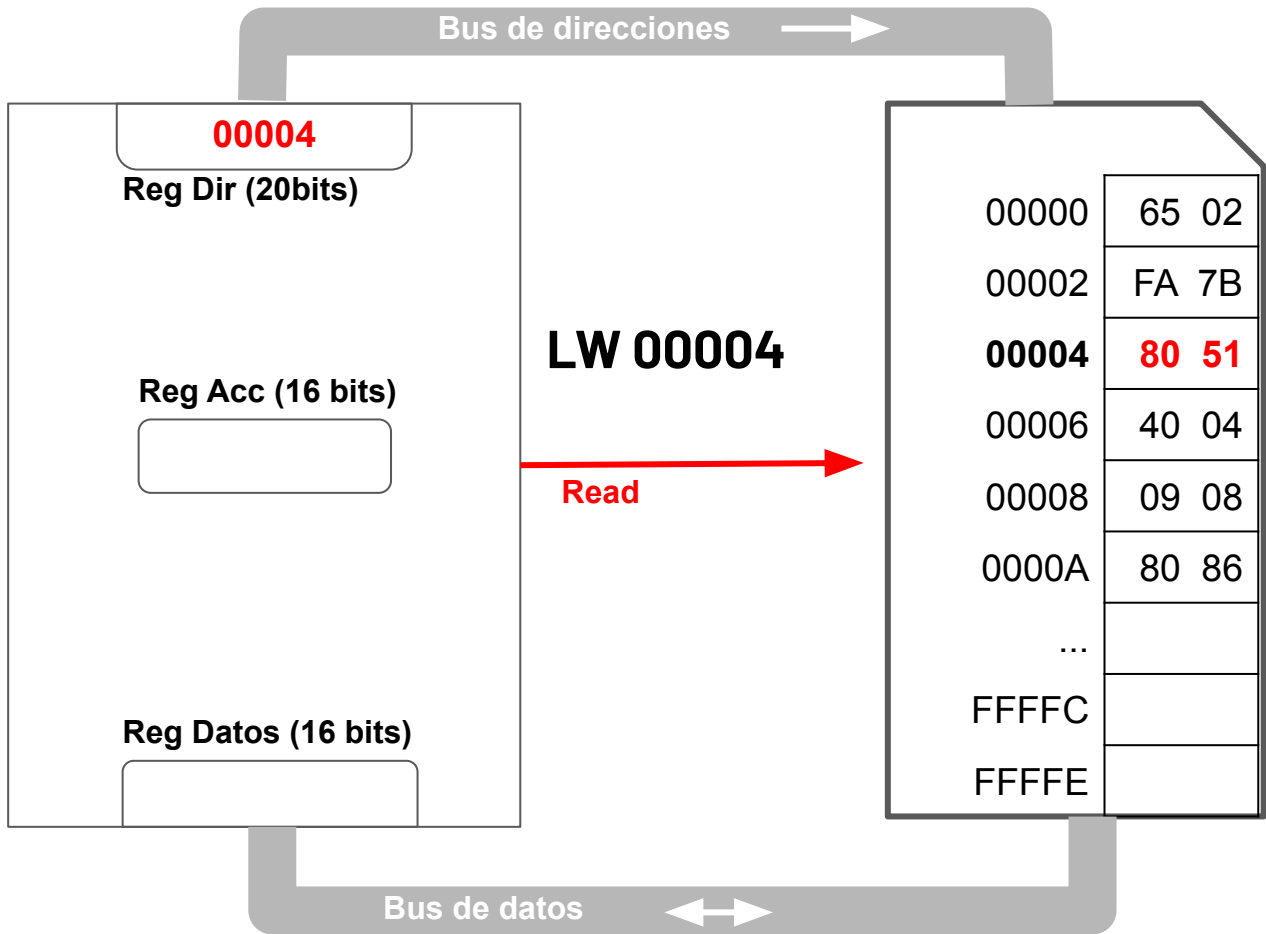
Las direcciones apuntan a un BYTE, NO a un word en memoria. Vemos que desde la CPU, cada WORD de memoria se incrementa de a dos (dado que hay 2 bytes por palabra).

# Arquitectura de 16 bits (dir al byte)



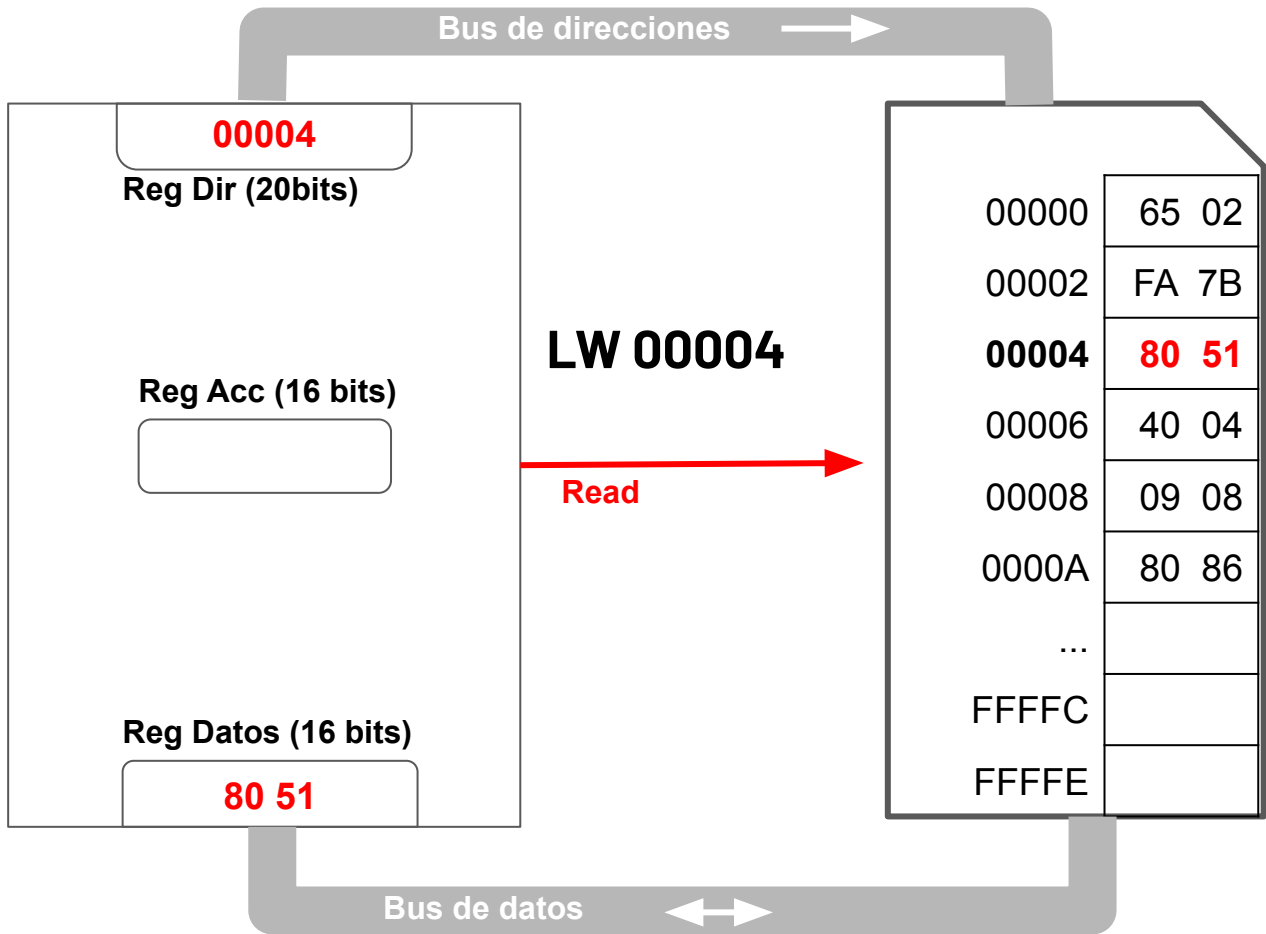
Cuando hablemos de direcciones vamos a SIEMPRE referirnos a direcciones apuntadas desde la CPU y no de la memoria propiamente dicha. Para leer un word usamos LW.

# Arquitectura de 16 bits (dir al byte)



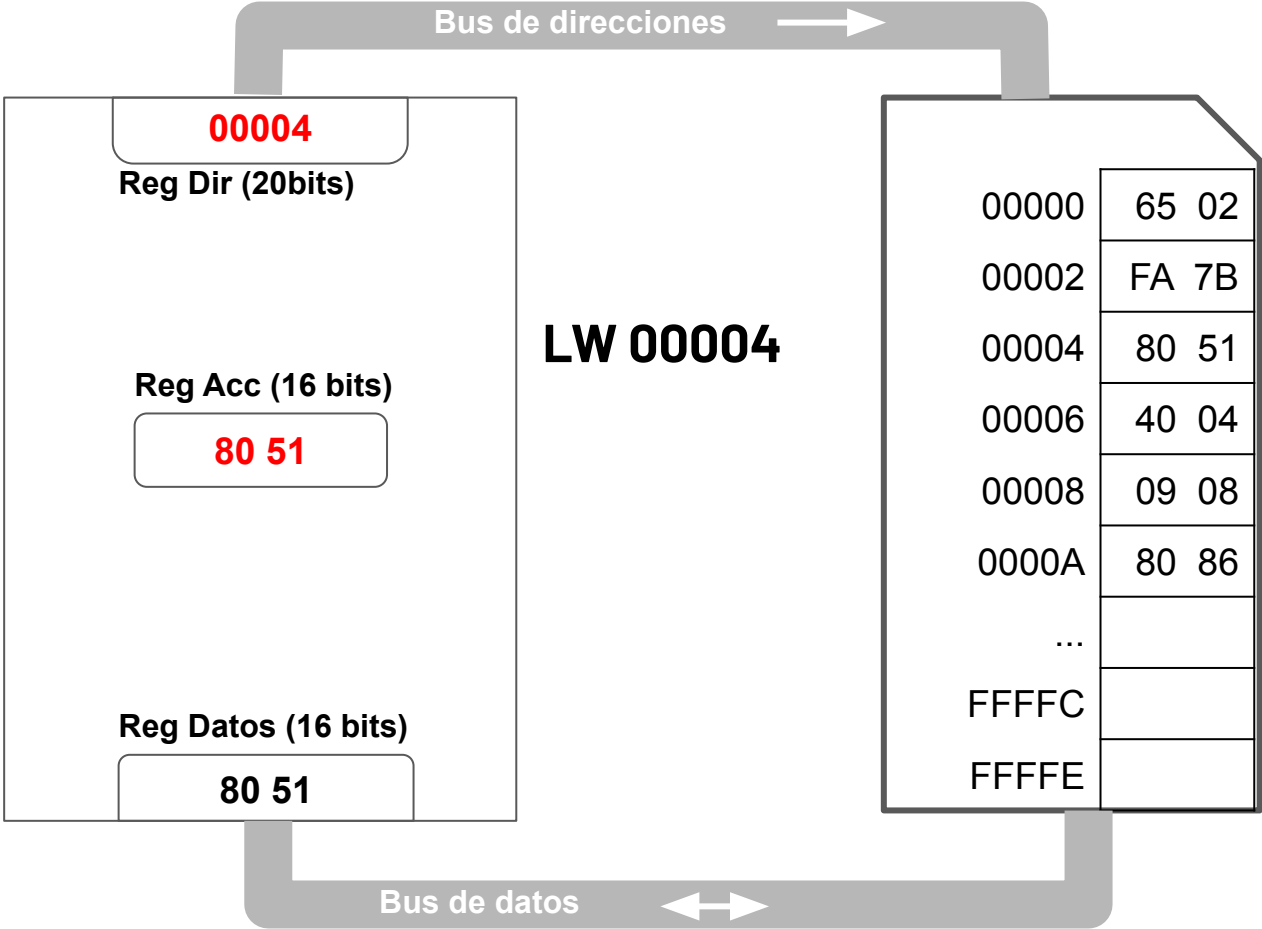
Las direcciones apuntan a un BYTE, NO a un word en memoria. Cada word en memoria consiste de 2 bytes. Ahora usamos LW dir para acceder a un word.

# Arquitectura de 16 bits (dir al byte)



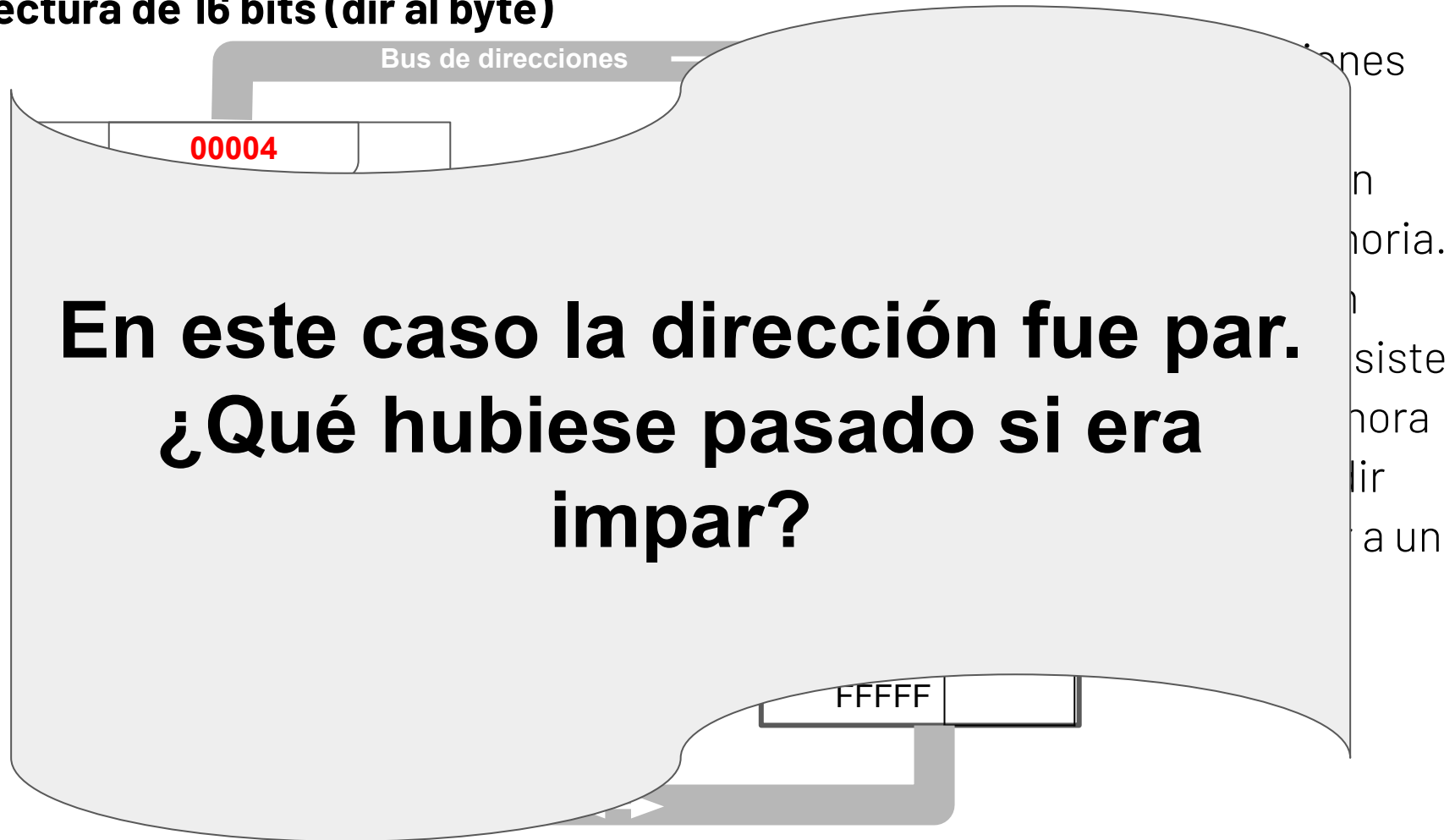
Las direcciones apuntan a un BYTE, NO a un word en memoria. Cada word en memoria consiste de 2 bytes. Ahora usamos LW dir para acceder a un word.

# Arquitectura de 16 bits (dir al byte)

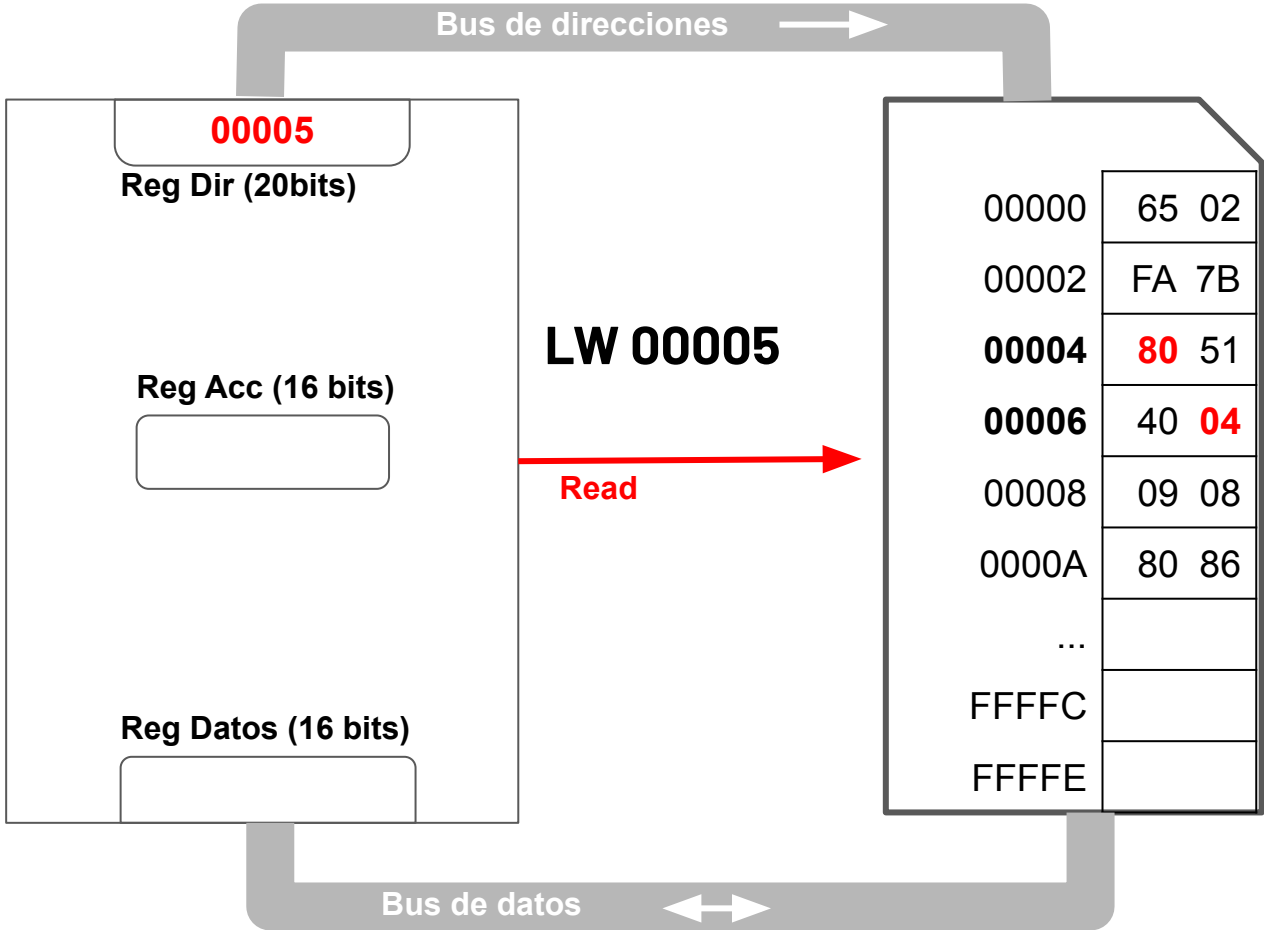


Las direcciones apuntan a un BYTE, NO a un word en memoria. Cada word en memoria consiste de 2 bytes. Ahora usamos LW dir para acceder a un word.

## Arquitectura de 16 bits (dir al byte)

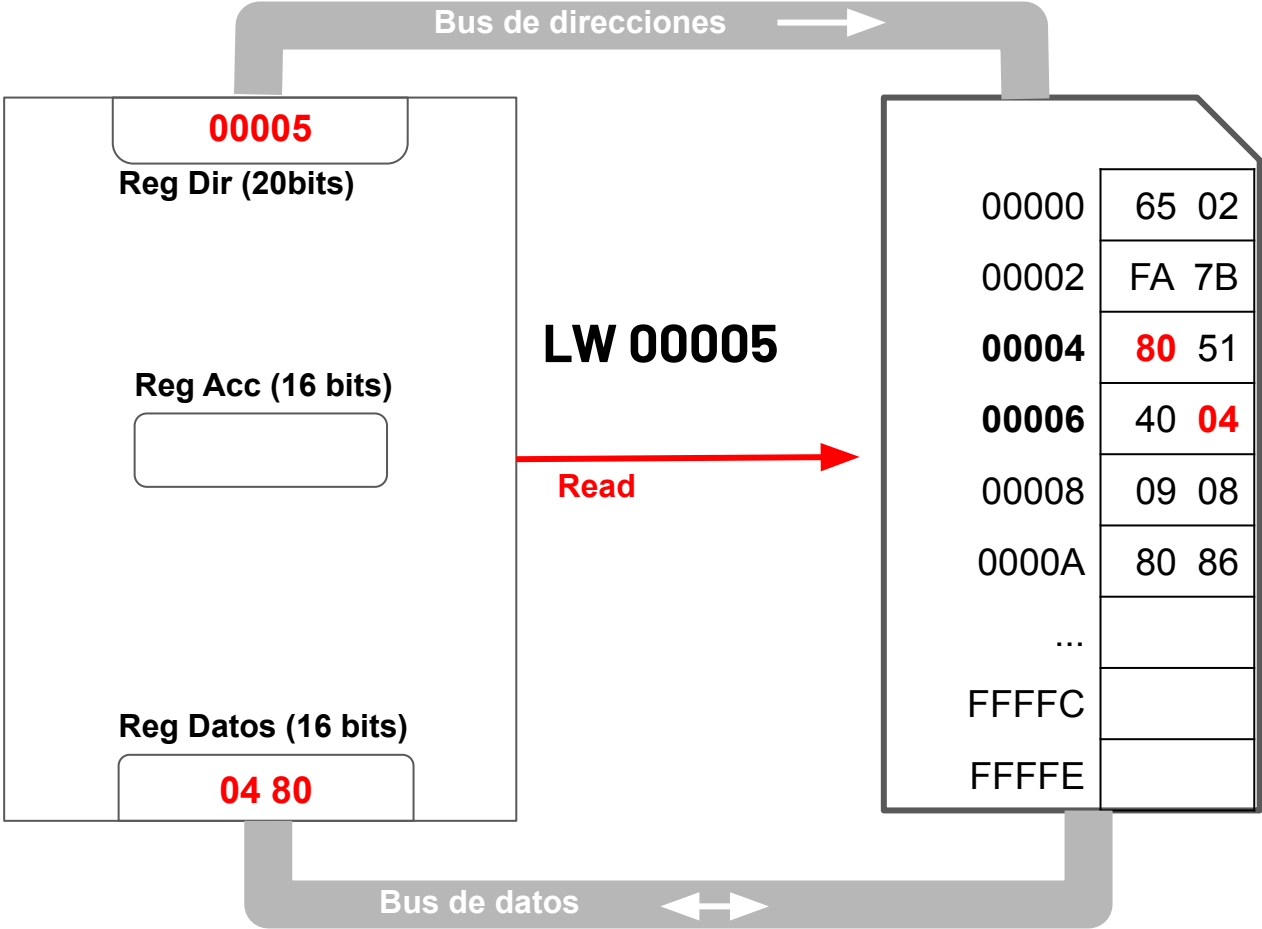


# Arquitectura de 16 bits (dir al byte)



Si la dir es impar, entonces apunta al byte más alto en un word de memoria y luego sigue con el byte más bajo del word siguiente.

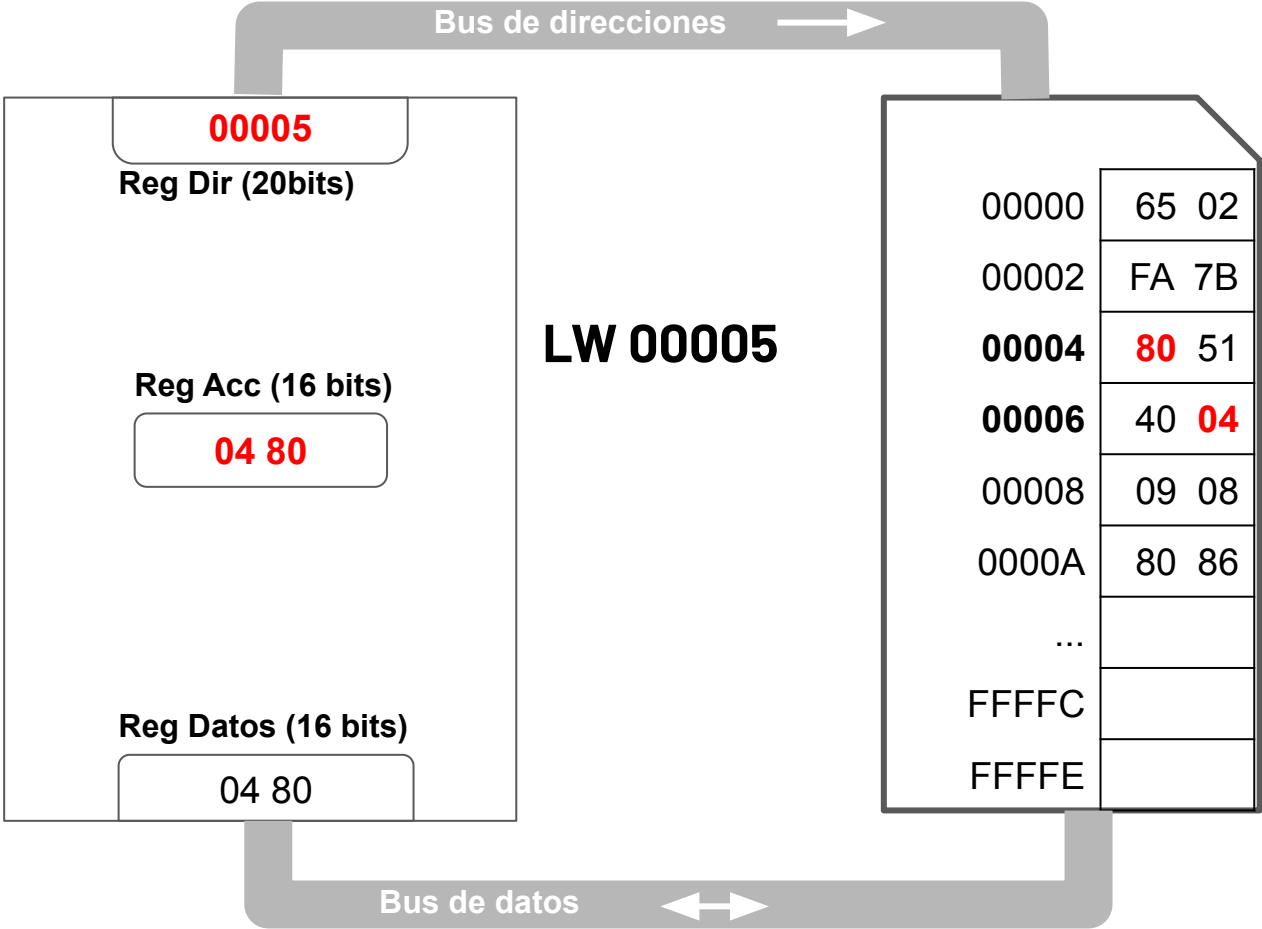
# Arquitectura de 16 bits (dir al byte)



Si la dir es impar, entonces apunta al byte más alto en un word de memoria y luego sigue con el byte más bajo del word siguiente.



# Arquitectura de 16 bits (dir al byte)



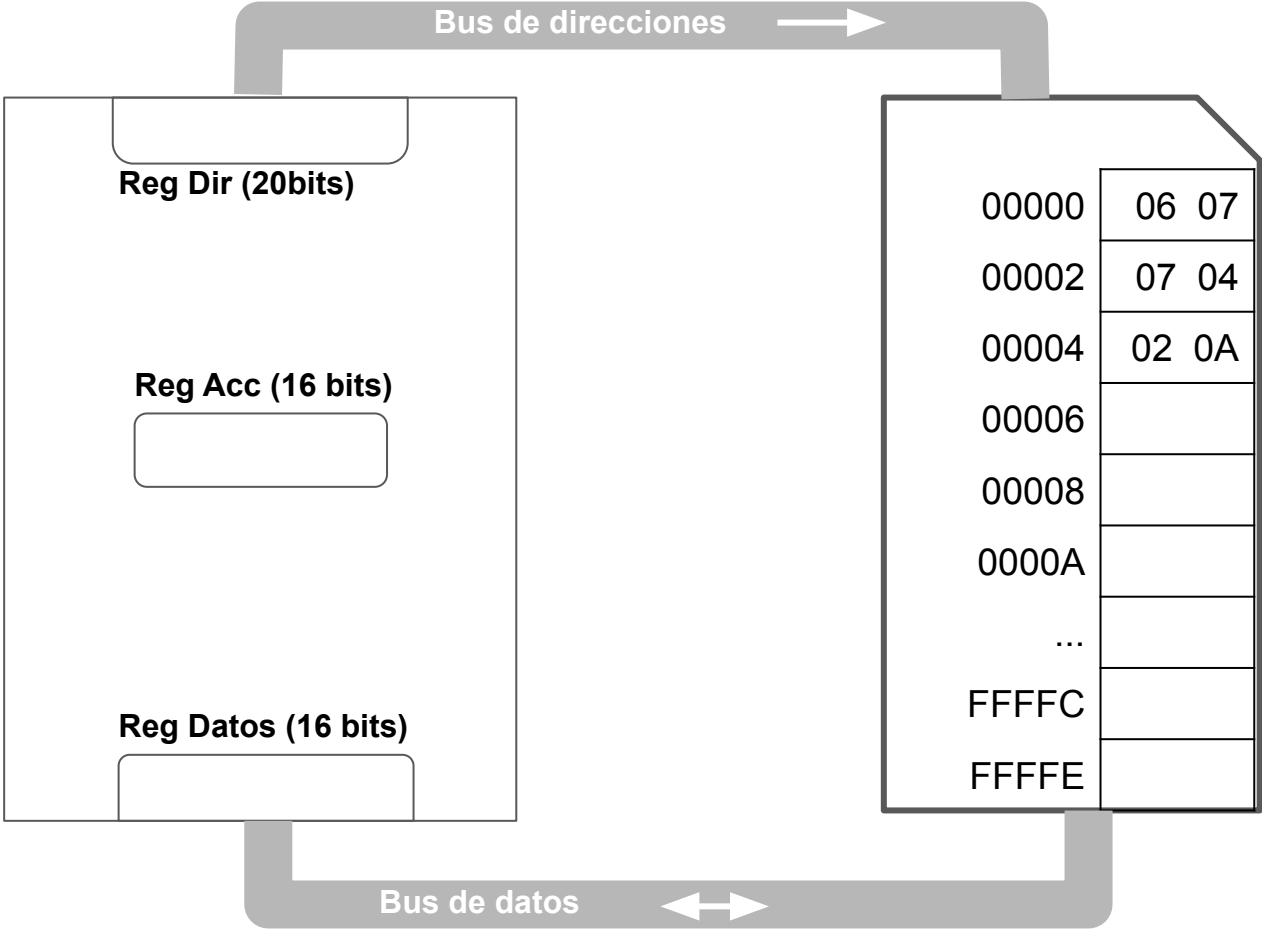
Este acceso es desalineado, ya que necesita acceder a dos words de memoria a la vez (00004 y 00006). NO todas las computadoras soportan este tipo de acceso. Son siempre más lentos que los accesos alineados.

# Direccionamiento al byte

Imaginemos que tenemos que almacenar la nota de los parciales de un curso en un vector en memoria (7,6,4,7,10,2). Si usamos una computadora de 16 bits, cada word es de 16 bits, permitiendo representar (sin signo) desde el 0 al 65535. Pero las notas van de 0 a 10, por ende estamos usando dos bytes para almacenar algo que solo utiliza un byte. Si en cada dir de memoria almacenamos dos notas, usamos la mitad de direcciones, aprovechando mucho mejor el espacio de memoria de la computadora. Pero ahora se necesita una forma simple de acceder a cada byte sin tener que leer la palabra completa... para eso usamos LH, o sea Load Half Word... que lee solo un byte de memoria.

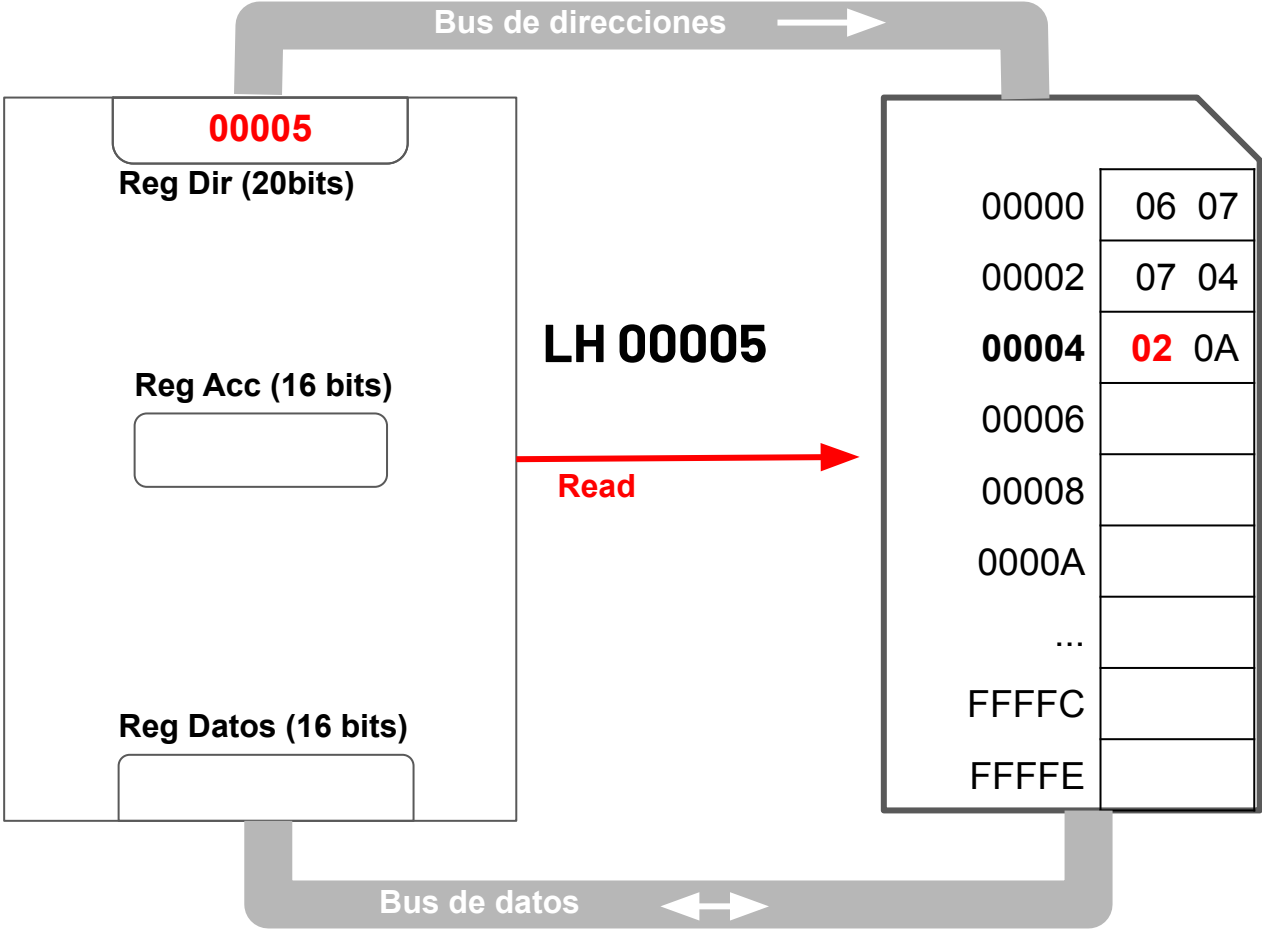
|       |      |       |       |
|-------|------|-------|-------|
| 00000 | 0007 | 00000 | 06 07 |
| 00001 | 0006 | 00002 | 07 04 |
| 00002 | 0004 | 00004 | 02 0A |
| 00003 | 0007 | 00006 |       |
| 00004 | 000A | 00008 |       |
| 00005 | 0002 | 0000A |       |
| ...   |      | ...   |       |
| FFFFE |      | FFFFC |       |
| FFFFF |      | FFFFE |       |

# Arquitectura de 16 bits (dir al byte)



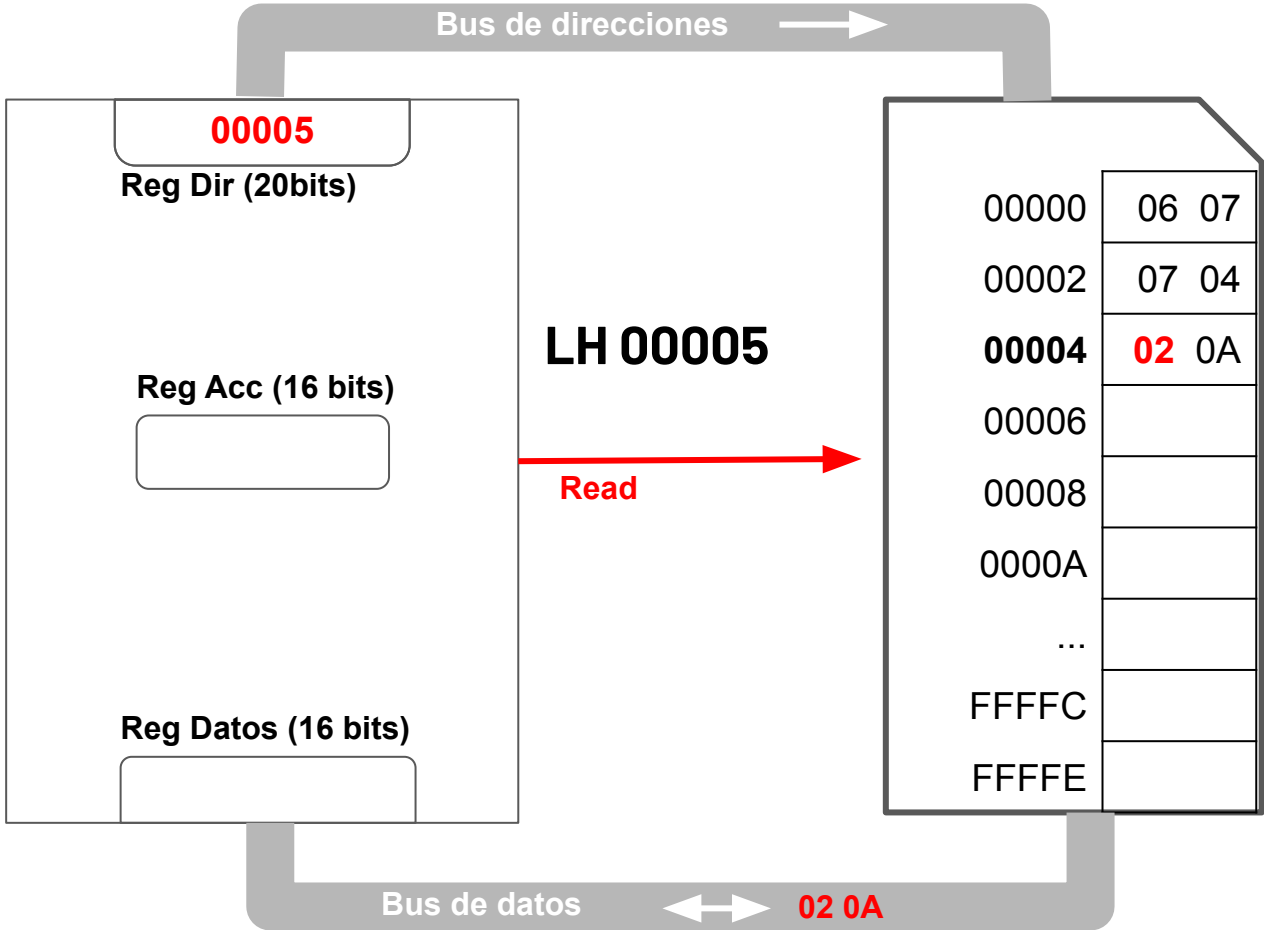
LH dir accede a un byte en cualquier dirección.  
Supongamos que queremos leer el 2 almacenado en 00005...

# Arquitectura de 16 bits (dir al byte)



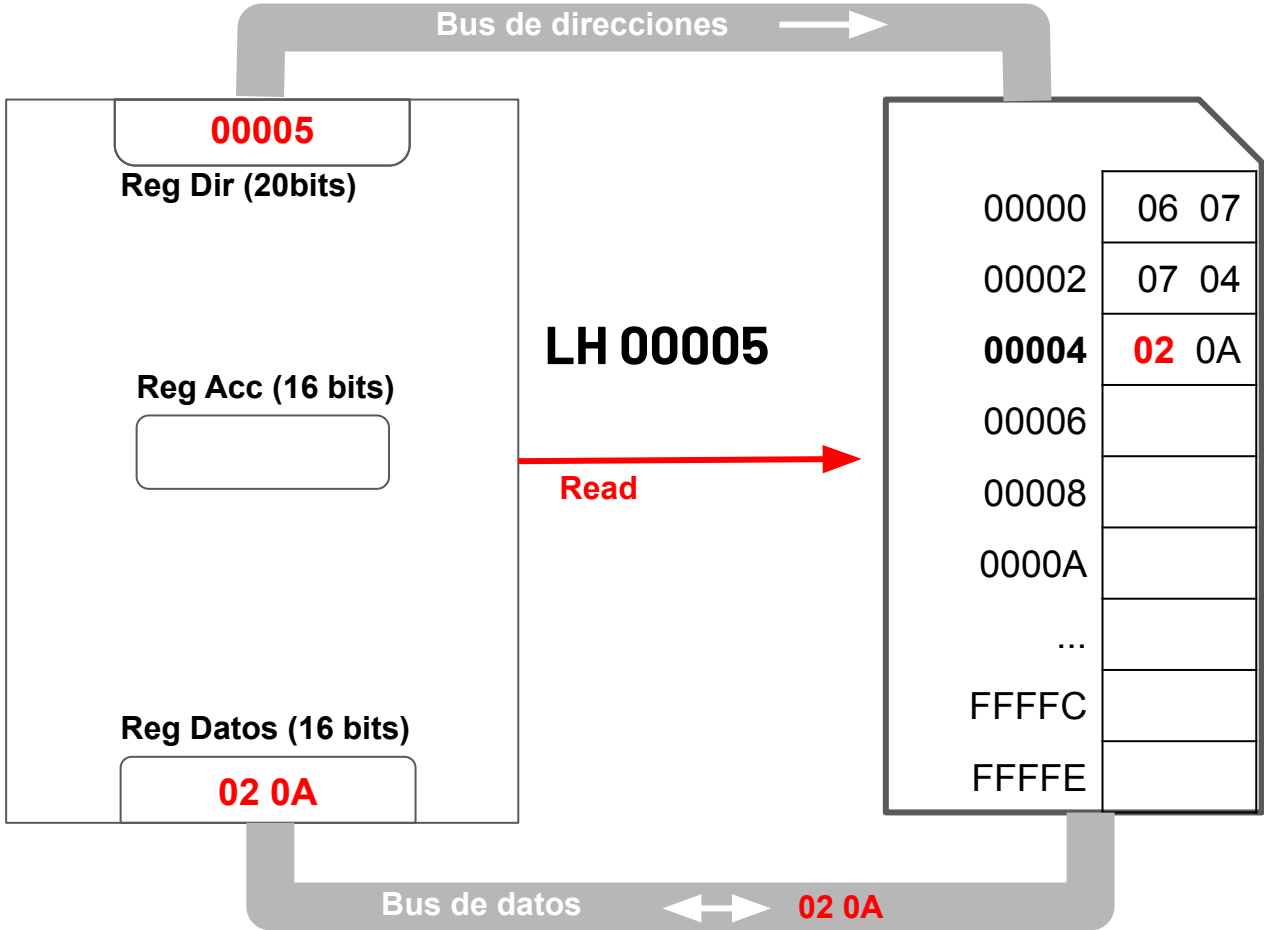
La dirección efectiva de memoria es 00004 (ya que 4 y 5 forman la misma palabra de memoria)

# Arquitectura de 16 bits (dir al byte)



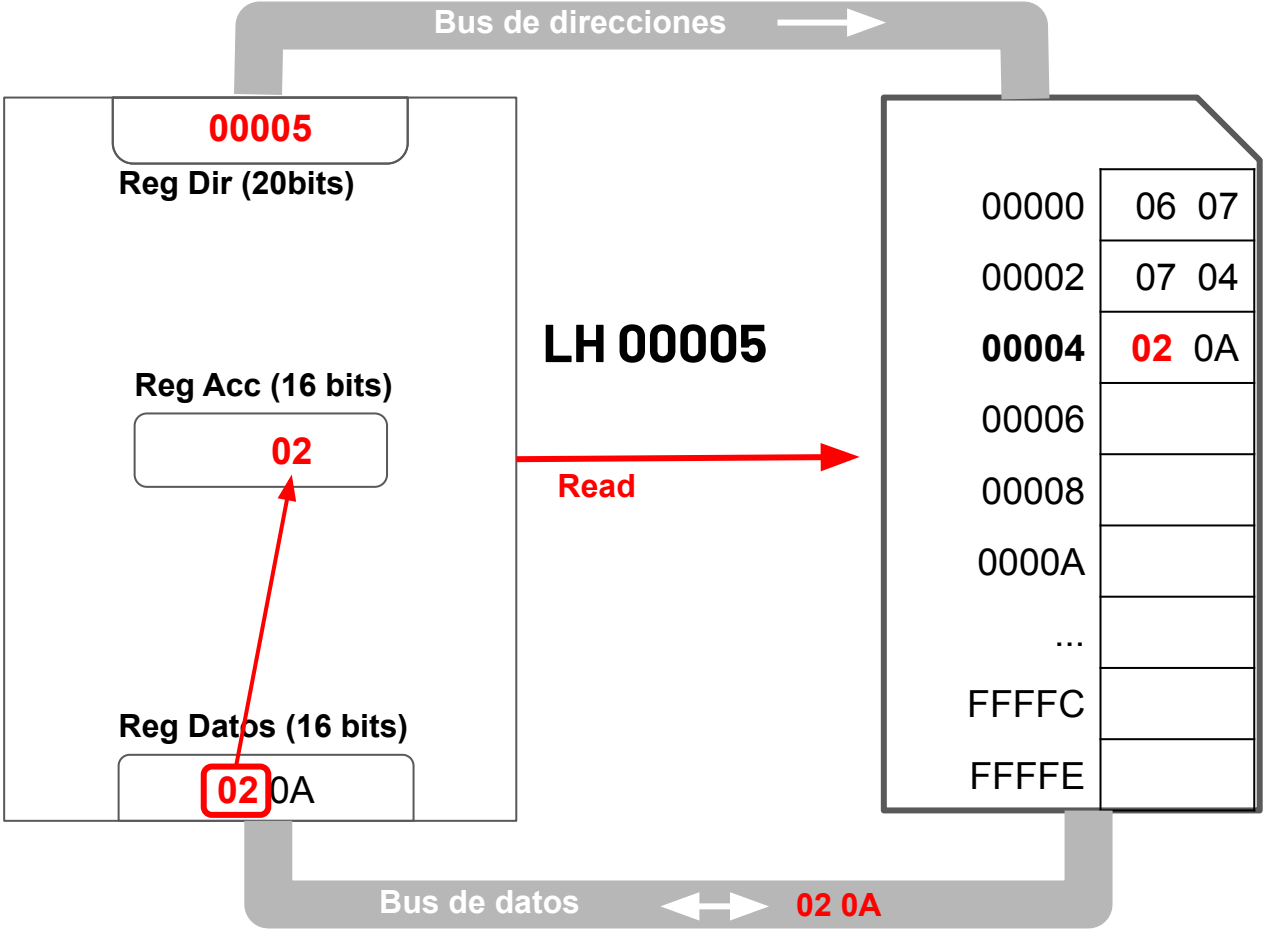
La memoria entrega el contenido de 00004 (02 0A) en el bus de datos.

# Arquitectura de 16 bits (dir al byte)



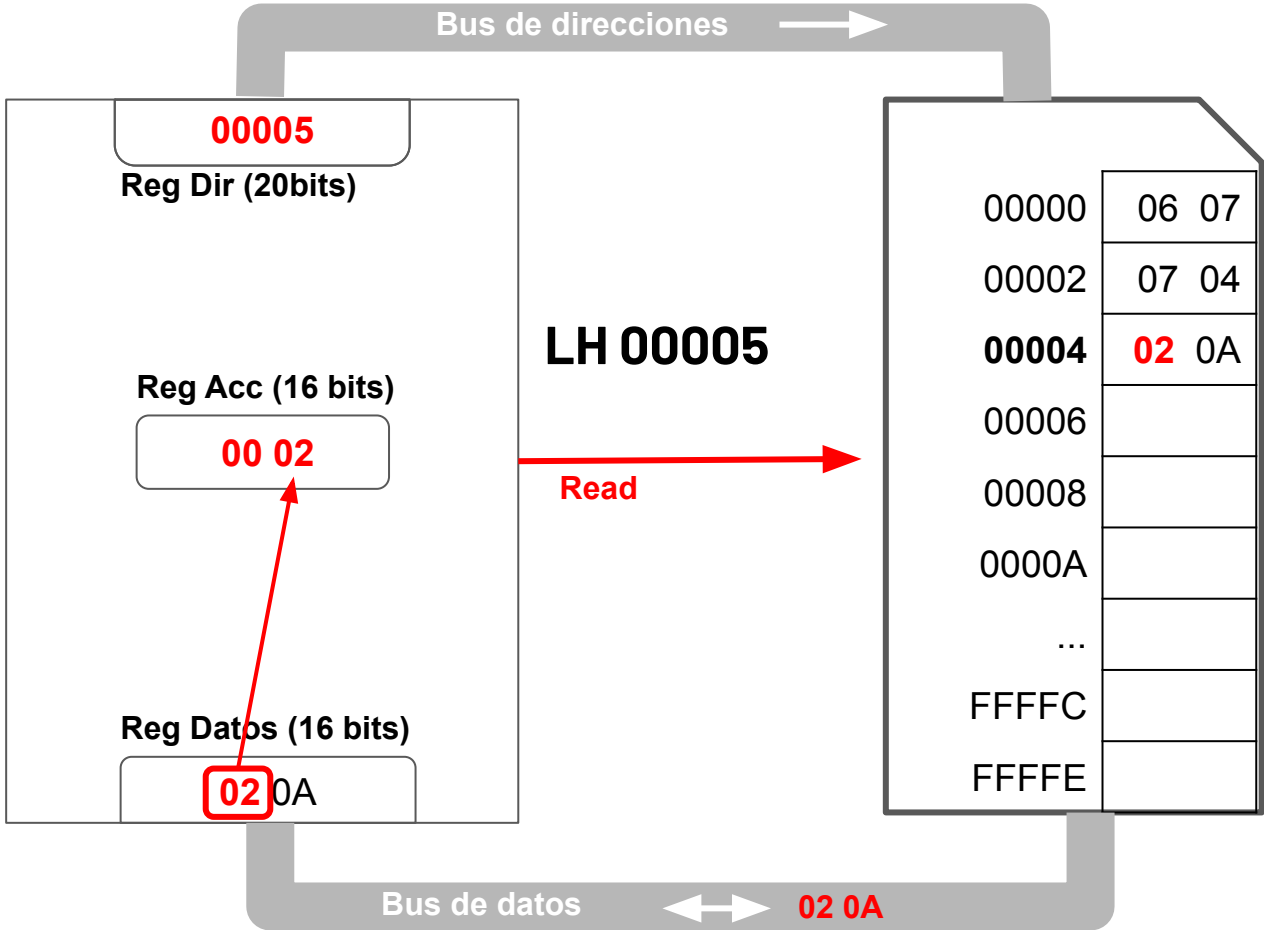
La CPU ahora tiene el valor, pero tiene que ajustarlo ya que LH requiere tomar solo un byte...

# Arquitectura de 16 bits (dir al byte)



Sabiendo que la dir es 00005 sabe que tiene que tomar la parte alta del word de 16 bits recibido. Copia ese valor a la parte BAJA del acc. Pero ahora necesita llenar los 8 bits altos del acc con algo...

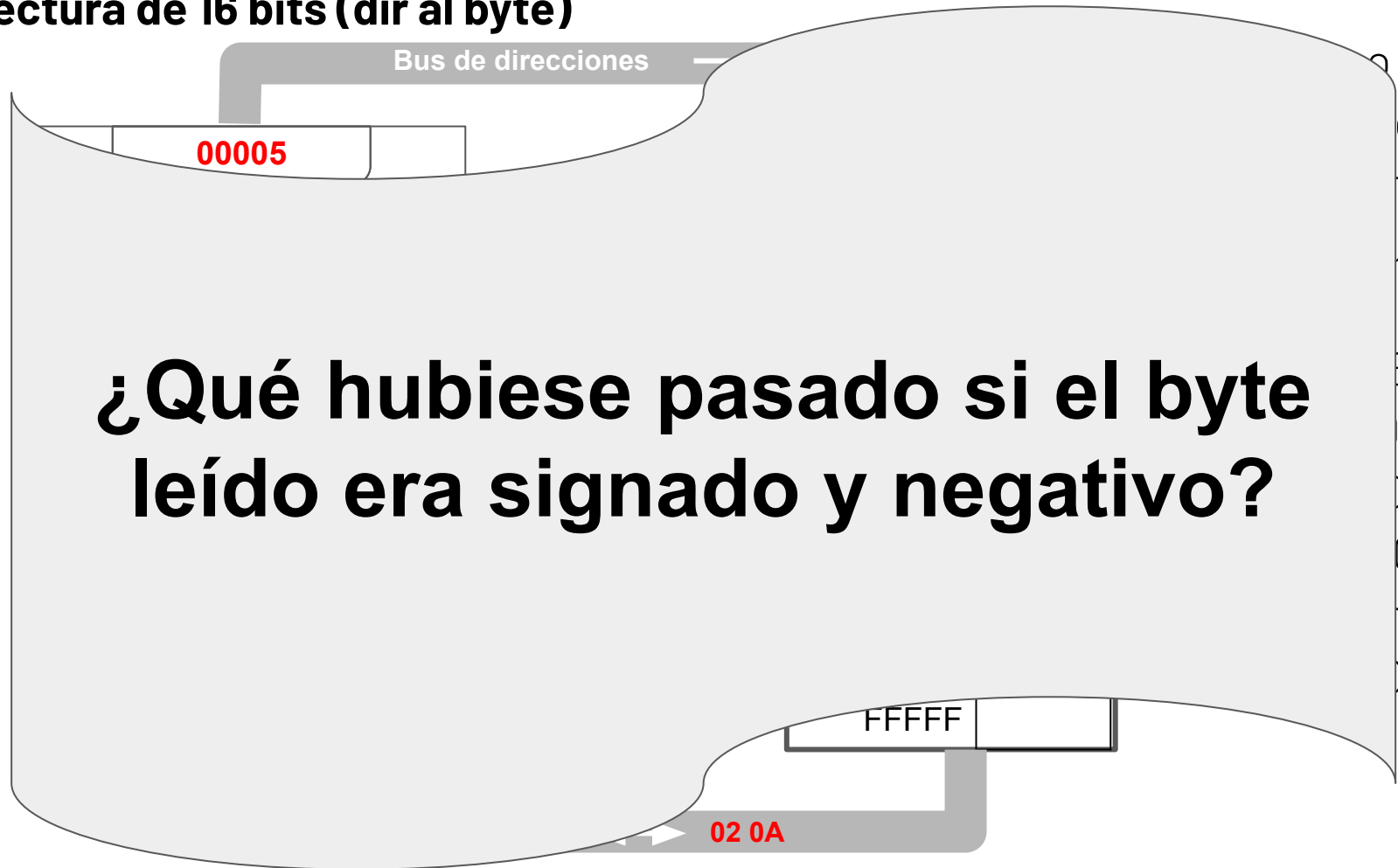
# Arquitectura de 16 bits (dir al byte)



En este caso 02 es positivo, por ende completa con 00. Pensemos que el número 2 (decimal) en 8 bits es 0000 0010. En 16 bits es 0000 0000 0010. Por ende en hexa el 02 pasa a 0002.

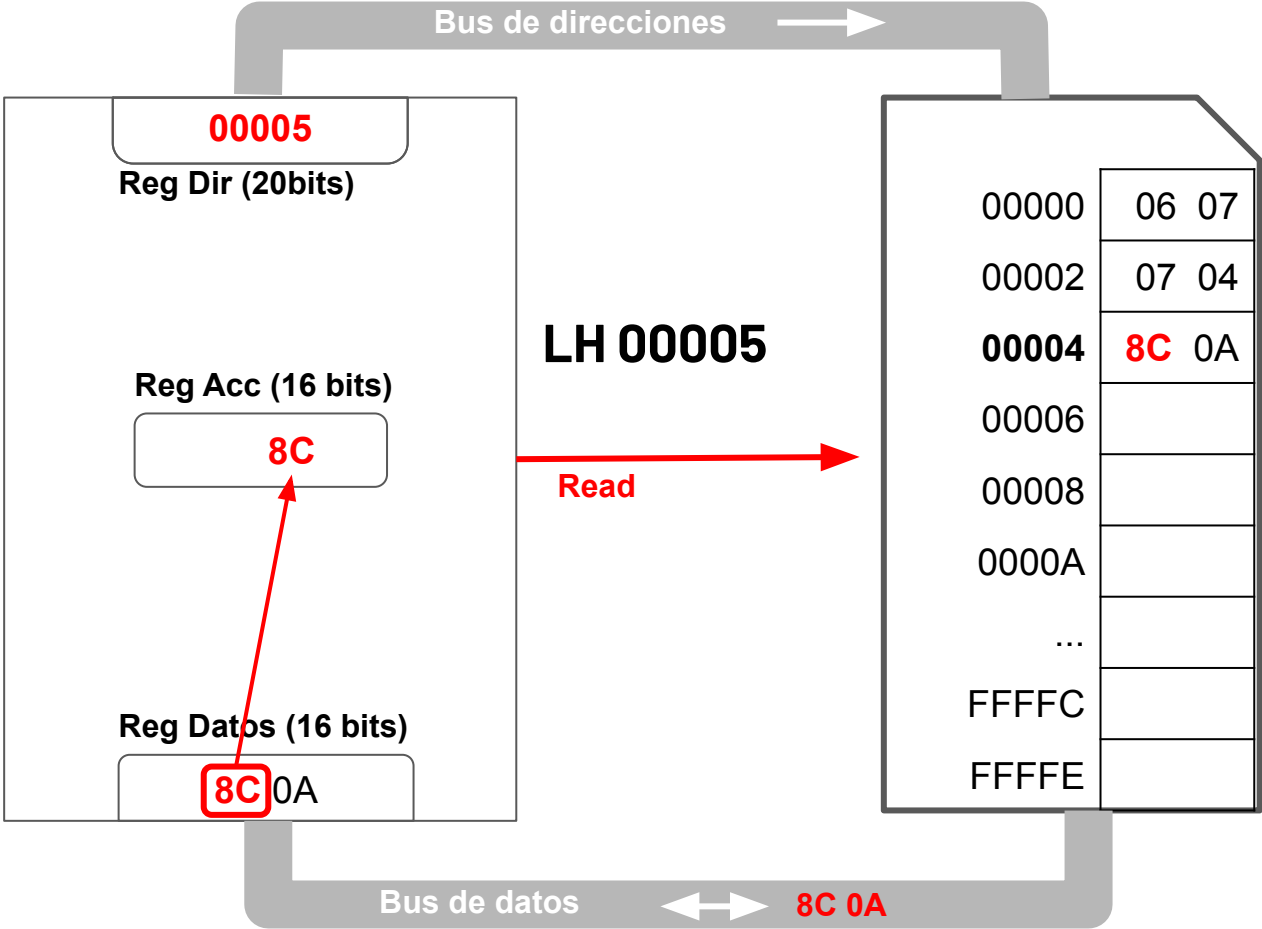


## Arquitectura de 16 bits (dir al byte)



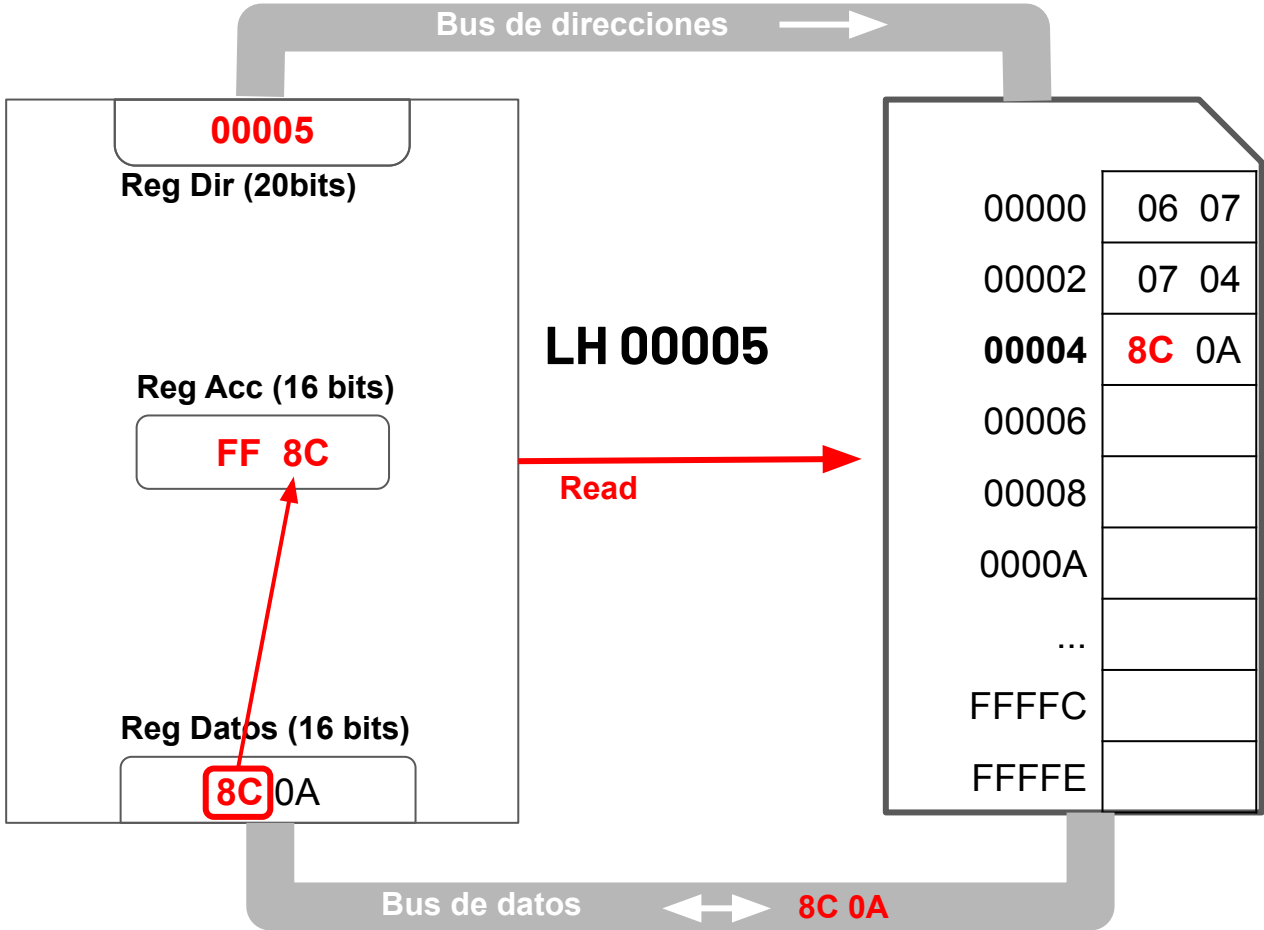
02 es  
ende  
n 00.  
e el  
  
3 bits  
. En  
00  
010.  
hexa  
0002.

# Arquitectura de 16 bits (dir al byte)



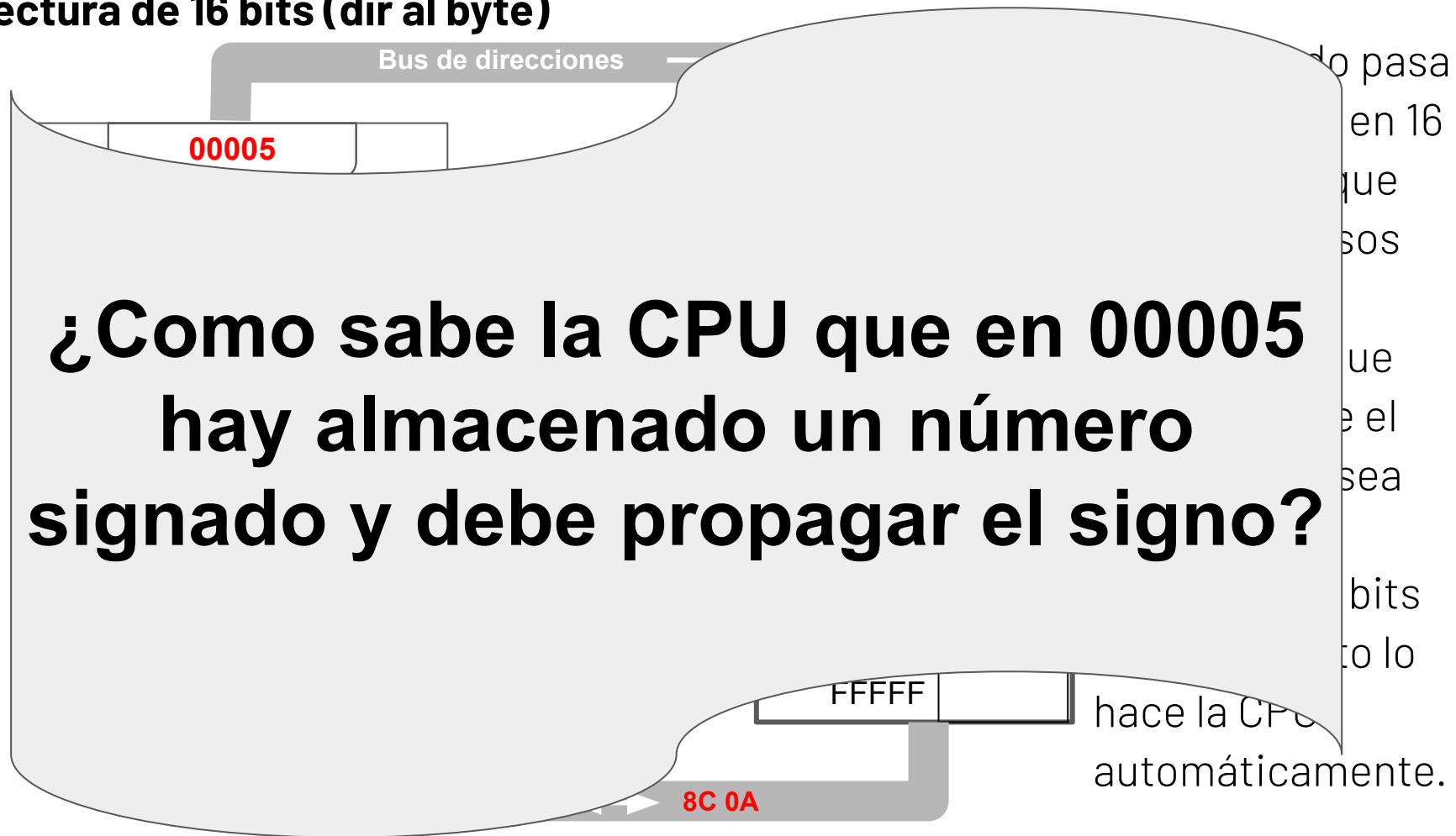
En esta caso 8C equivale al binario 1000 1100. Vemos que el bit más significativo es 1, por ende negativo (equivale a -116). Se propaga el signo hasta cubrir los 16 bits...

# Arquitectura de 16 bits (dir al byte)

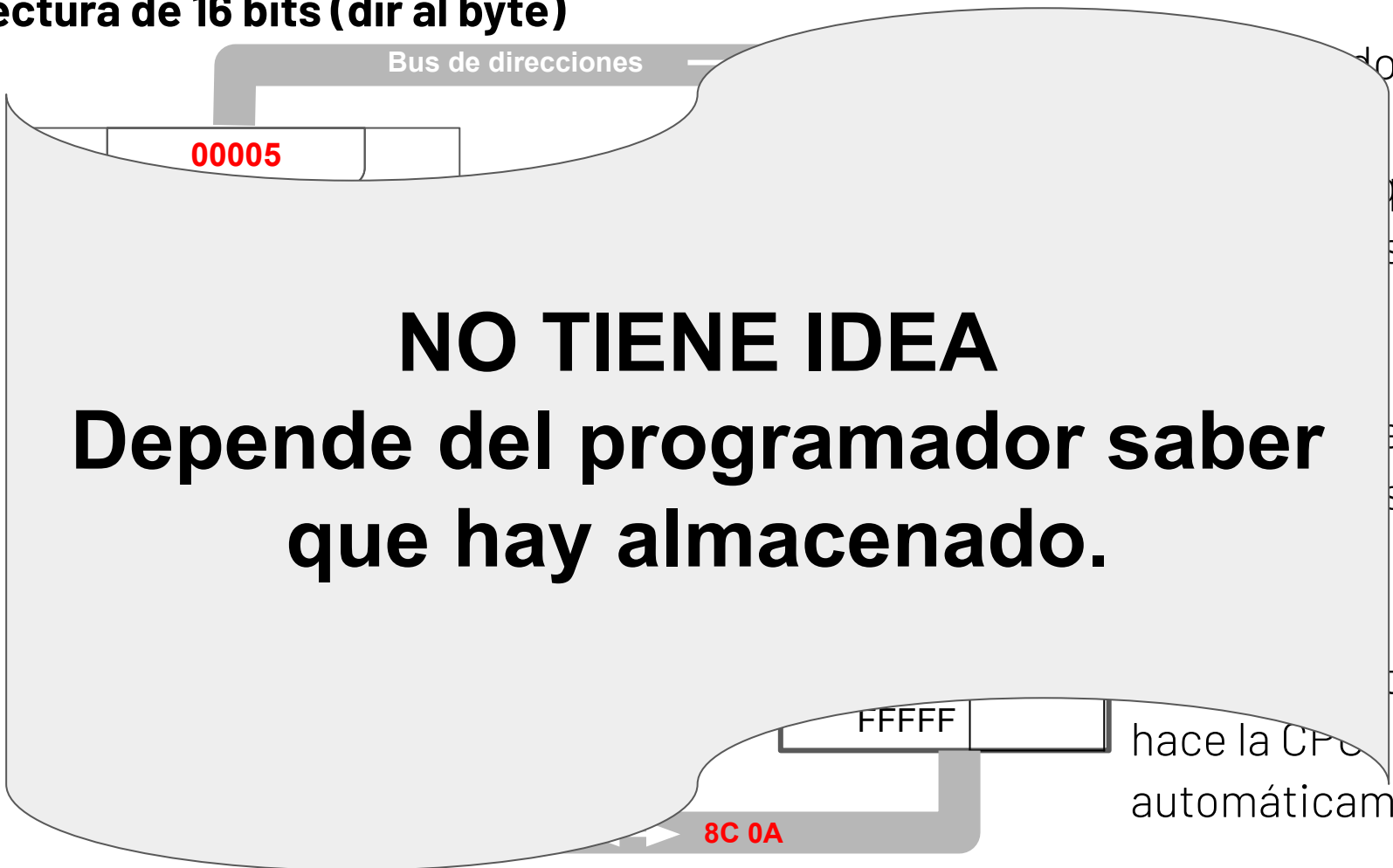


El 8C signado pasa a ser el FF8C en 16 bits. Vemos que en ambos casos (positivo o negativo) lo que ocurre es que el bit de signo (sea cero o uno) completa los bits faltantes. Esto lo hace la CPU automáticamente.

## Arquitectura de 16 bits (dir al byte)



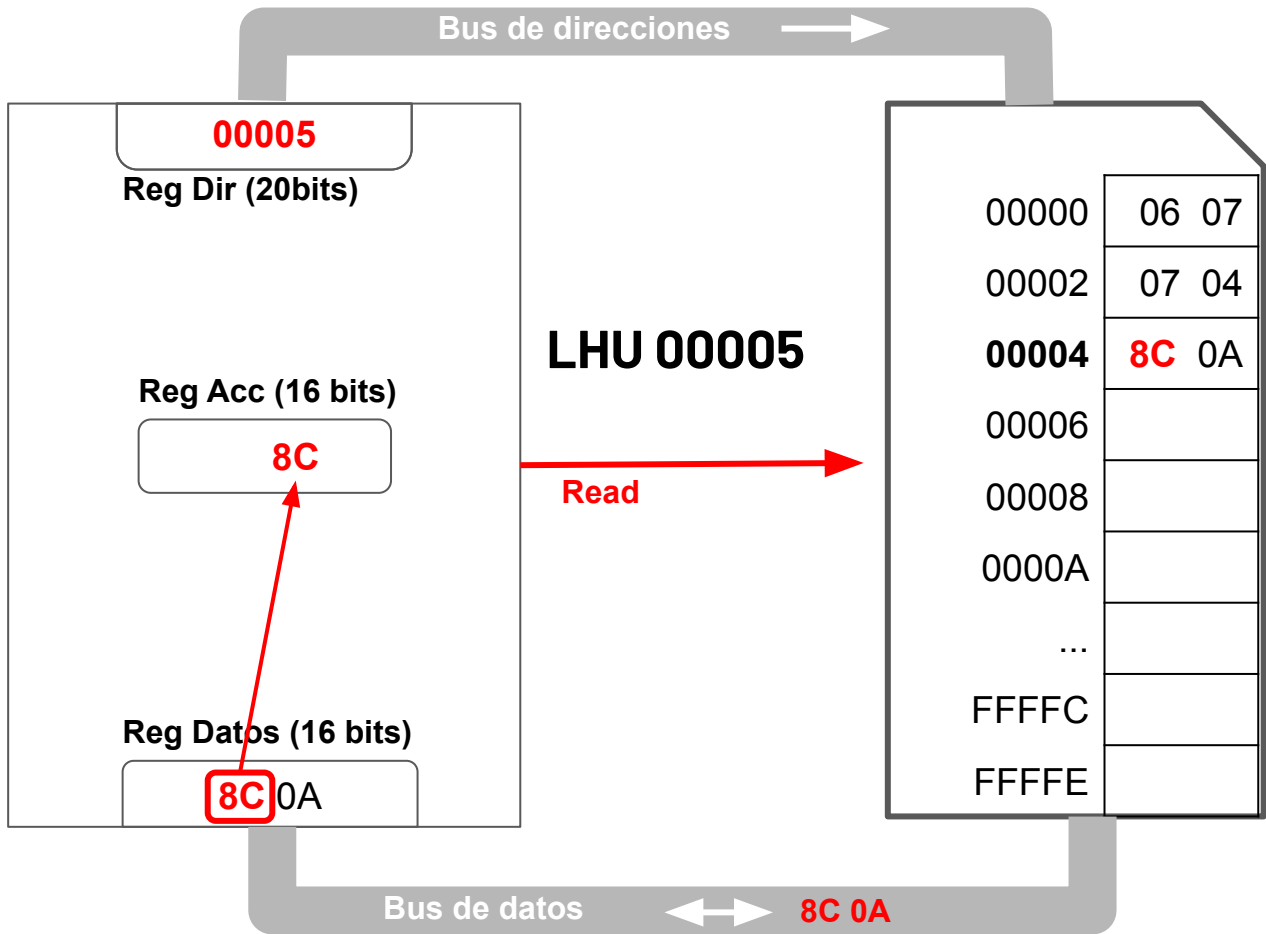
# Arquitectura de 16 bits (dir al byte)



**NO TIENE IDEA**  
**Depende del programador saber**  
**que hay almacenado.**

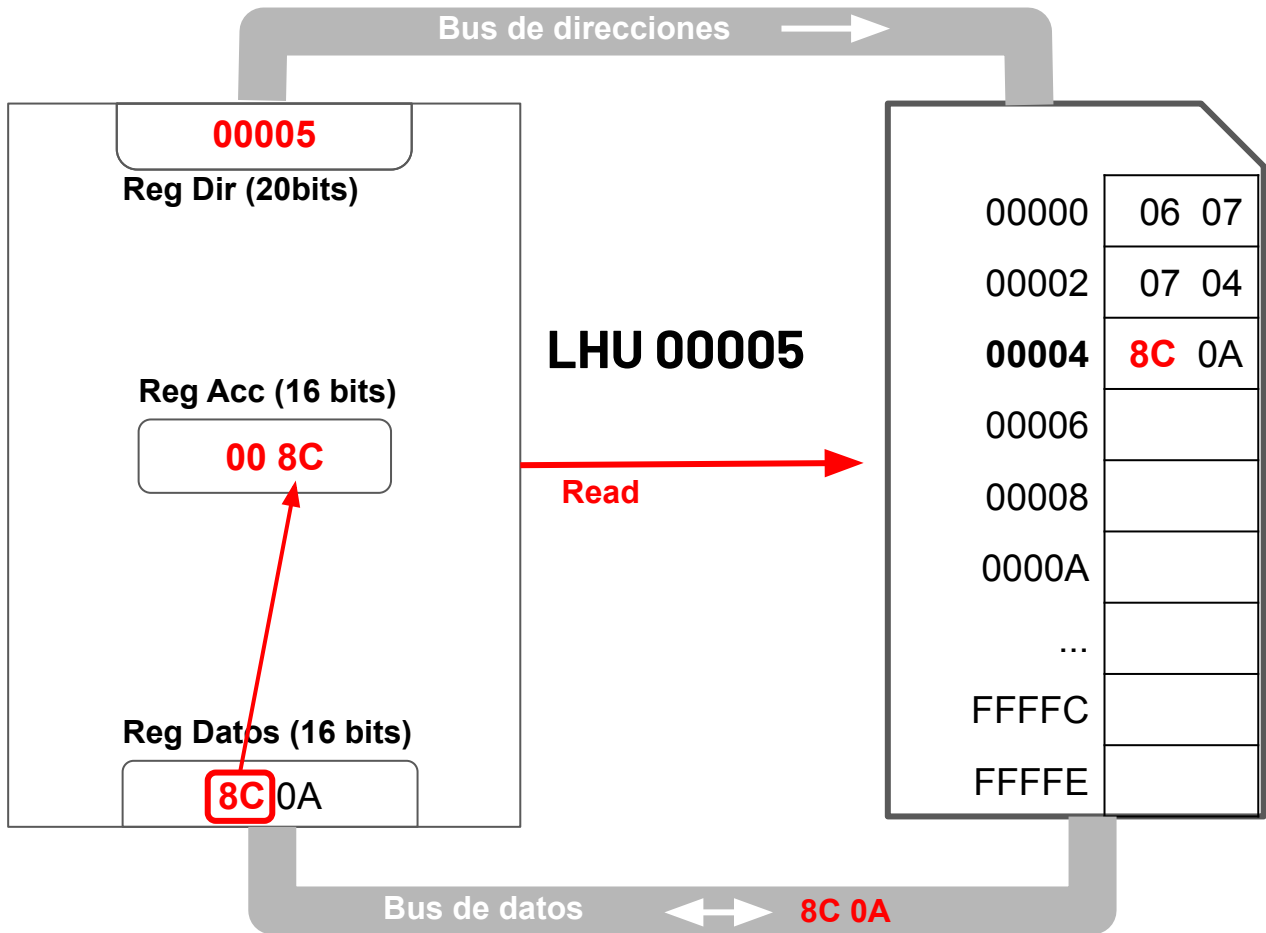
hace la CPU  
automáticamente.

# Arquitectura de 16 bits (dir al byte)



Existe LHU , Load Half Word Unsigned que trata el byte leído como sin signo. En este caso NO propaga el bit de signo sino que SIEMPRE completa con ceros.

# Arquitectura de 16 bits (dir al byte)



Si estamos trabajando con números sin signo (unsigned) entonces usamos las instrucciones unsigned (LHU).

# Direccionamiento al byte en arquitectura de 16 bits

Debemos pensar entonces que cada palabra de memoria está compuesta de 2 bytes. Si la dirección es PAR apunta al byte BAJO mientras que si es IMPAR apunta al byte ALTO.

|       |       |       |
|-------|-------|-------|
| 00000 | 00001 | 00000 |
| 00002 | 00003 | 00002 |
| 00004 | 00005 | 00004 |
| 00006 | 00007 | 00006 |
| 00008 | 00009 | 00008 |
| 0000A | 0000B | 0000A |
| ...   |       |       |
| FFFFC | FFFFD | FFFFC |
| FFFFE | FFFFF | FFFFE |



# Direccionamiento al byte en arquitectura de 16 bits

Debemos pensar entonces que cada palabra de memoria está compuesta de 2 bytes. Si la dirección es PAR apunta al byte BAJO mientras que si es IMPAR apunta al byte ALTO. Si accedemos a la memoria con LW (Load Word) y la dirección es PAR, el acceso es alineado.

Ej: **LW 00004 .. Alineado**, accede a 00004 y 00005 ambos bytes conforman la misma palabra de memoria (00004).

|       |              |              |
|-------|--------------|--------------|
| 00000 | 00001        | 00000        |
| 00002 | 00003        | 00002        |
| 00004 | <b>00005</b> | <b>00004</b> |
| 00006 | 00007        | 00006        |
| 00008 | 00009        | 00008        |
| 0000A | 0000B        | 0000A        |
| ...   |              |              |
| FFFFC | FFFFD        | FFFFC        |
| FFFFE | FFFFF        | FFFFE        |

# Direccionamiento al byte en arquitectura de 16 bits

Debemos pensar entonces que cada palabra de memoria está compuesta de 2 bytes. Si la dirección es PAR apunta al byte BAJO mientras que si es IMPAR apunta al byte ALTO. Si accedemos a la memoria con LW (Load Word) y la dirección es PAR, el acceso es alineado. Si es IMPAR es desalineado.

Ej: **LW 00005 ..NO Alineado**, accede a 00005 y 00006 que pertenecen a palabras de memoria distintas.

|       |              |              |
|-------|--------------|--------------|
| 00000 | 00001        | 00000        |
| 00002 | 00003        | 00002        |
| 00004 | <b>00005</b> | 00004        |
| 00006 | 00007        | <b>00006</b> |
| 00008 | 00009        | 00008        |
| 0000A | 0000B        | 0000A        |
| ...   |              |              |
| FFFFC | FFFFD        | FFFFC        |
| FFFFE | FFFFF        | FFFFE        |

# Direccionamiento al byte en arquitectura de 16 bits

Debemos pensar entonces que cada palabra de memoria está compuesta de 2 bytes. Si la dirección es PAR apunta al byte BAJO mientras que si es IMPAR apunta al byte ALTO. Si accedemos a la memoria con LW (Load Word) y la dirección es PAR, el acceso es alineado. Si es IMPAR es desalineado.

En el caso de acceder a solo un byte (LH o LHU), sea PAR o IMPAR no importa, ya que solo se accede a un byte, por ende siempre va a estar alineado... es decir.. siempre va a encontrarse el dato en una única palabra de memoria.

EJ: LH 00005 ... Alineado.

|       |              |       |
|-------|--------------|-------|
| 00000 | 00001        | 00000 |
| 00002 | 00003        | 00002 |
| 00004 | <b>00005</b> | 00004 |
| 00006 | 00007        | 00006 |
| 00008 | 00009        | 00008 |
| 0000A | 0000B        | 0000A |
| ...   |              |       |
| FFFFC | FFFFD        | FFFFC |
| FFFFE | FFFFF        | FFFFE |

| 16 bits | Dirección | High(1) | Low(0) |
|---------|-----------|---------|--------|
|         | 0000      | 0001    | 0000   |
|         | 0002      | 0003    | 0002   |
|         | 0004      | 0005    | 0004   |
|         | 0006      | 0007    | 0006   |
|         | 0008      | 0009    | 0008   |
|         | 000A      | 000B    | 000A   |
|         | 000C      | 000D    | 000C   |
|         | 000E      | 000F    | 000E   |
|         | 0010      | 0011    | 0010   |
|         | 0012      | 0013    | 0012   |
|         | ...       |         |        |
|         | FFFE      | FFFF    | FFFE   |

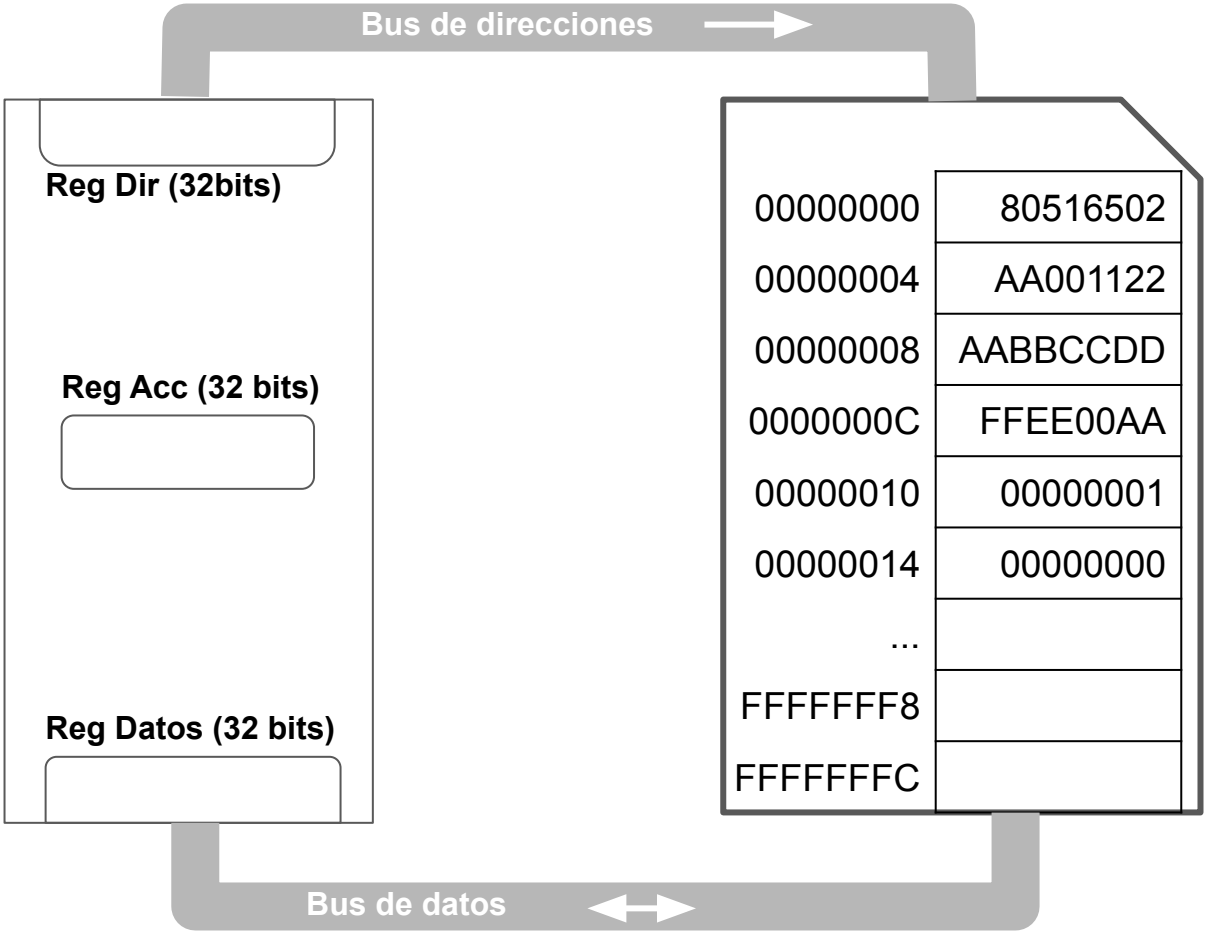
Las palabras (words) son de 2 bytes. Usar LW accede a 2 bytes en memoria. Si la dir termina en número PAR (0,2,4,6,8,A,C,E) entonces el acceso está alineado a 16 bits.

Se puede acceder a media palabra (Half Word) utilizando LH. En ese caso, se accede a un byte (siempre alineado). El valor leído se expande a 16 bits propagando el signo. En el caso de no querer propagar el signo se utiliza LHU (Load Half Word Unsigned) que completa el byte más significativo con 00. Ej:

LW 0x0002 (Alineado)

LW 0x0009 (No Alineado)

# Arquitectura de 32 bits (dir al byte)

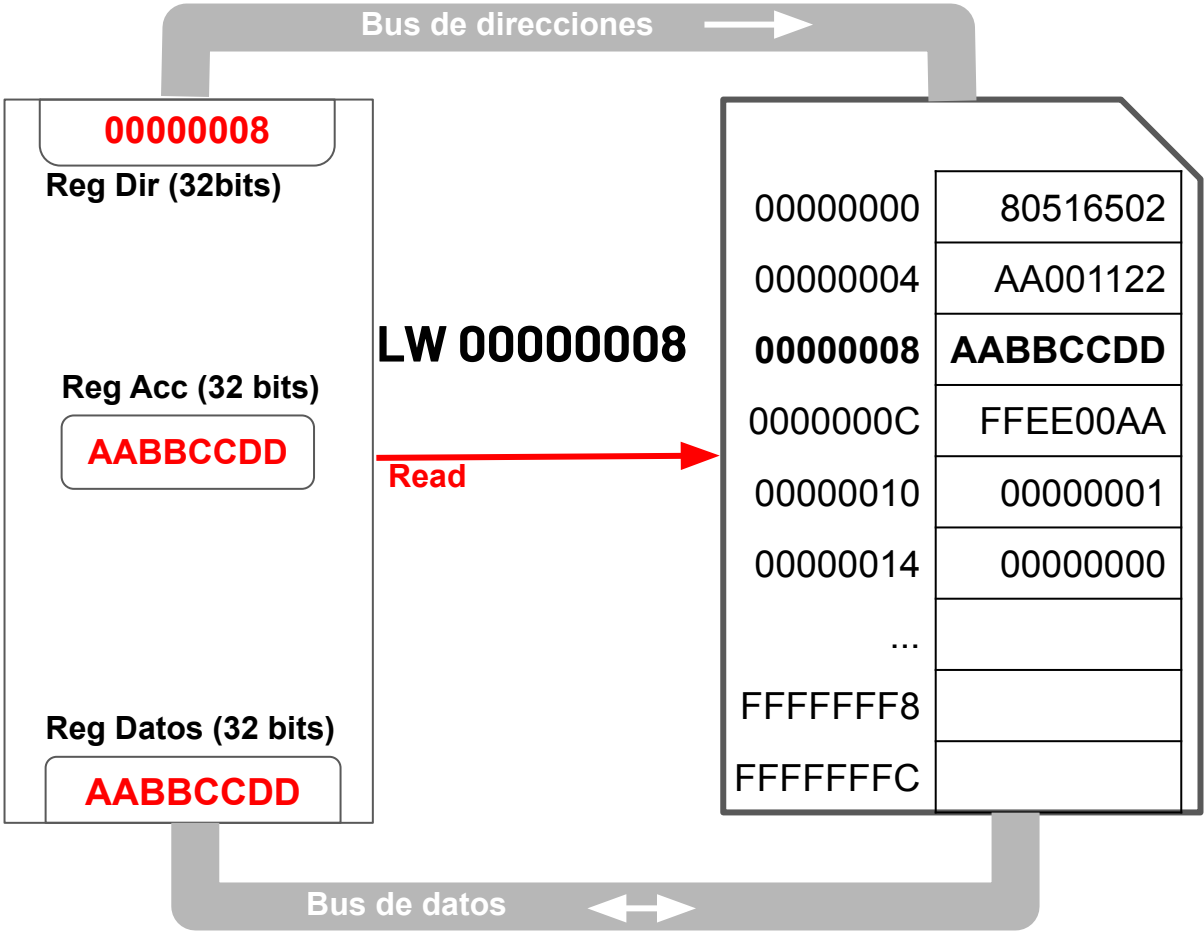


Si ahora tenemos word de 32 bits (4 bytes), también direccionamos al byte. Ahora la unidad es:

- Word 32 bits
- HalfW 16 bits
- Byte 8 bits

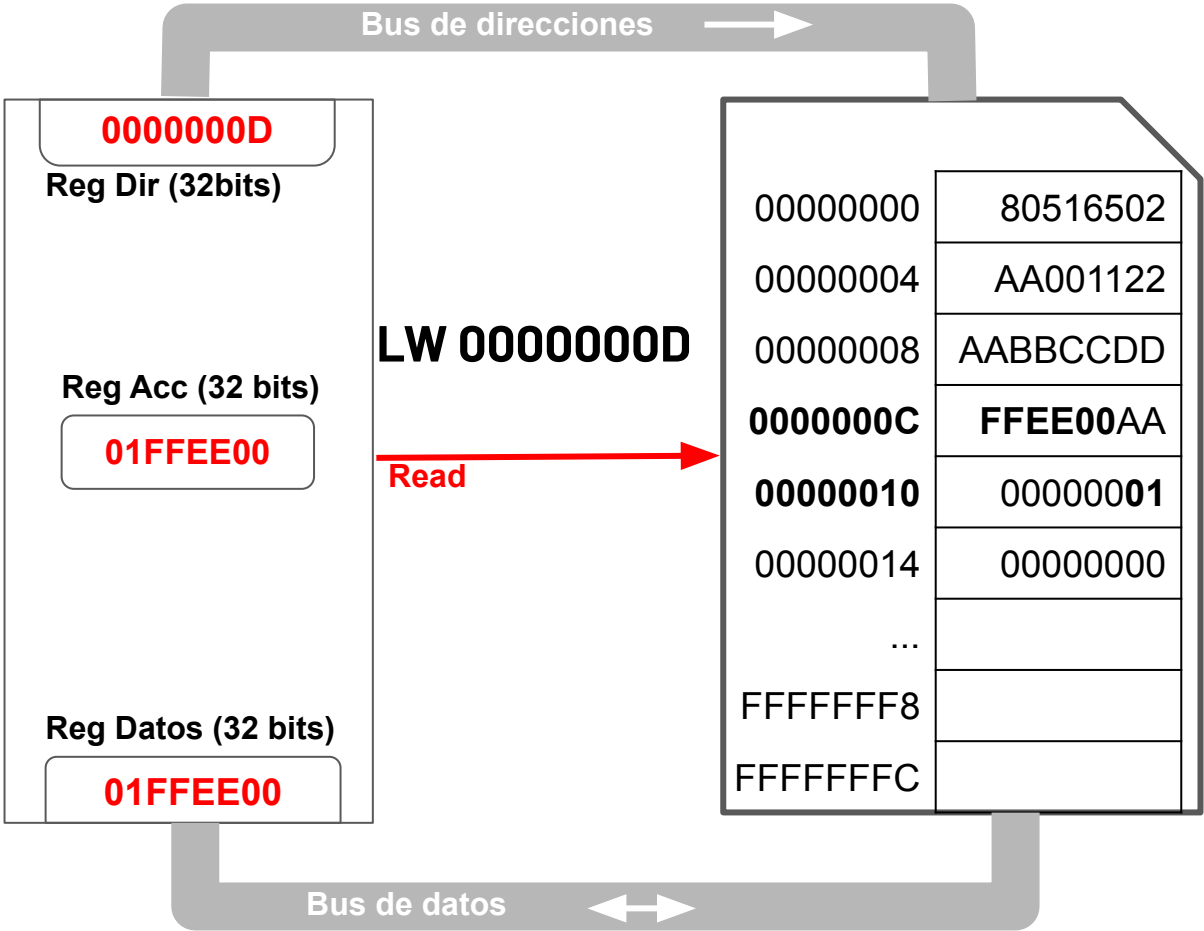
Con instrucciones LW, LH, LB signadas y LHU, LBU sin signo.

# Arquitectura de 32 bits (dir al byte)



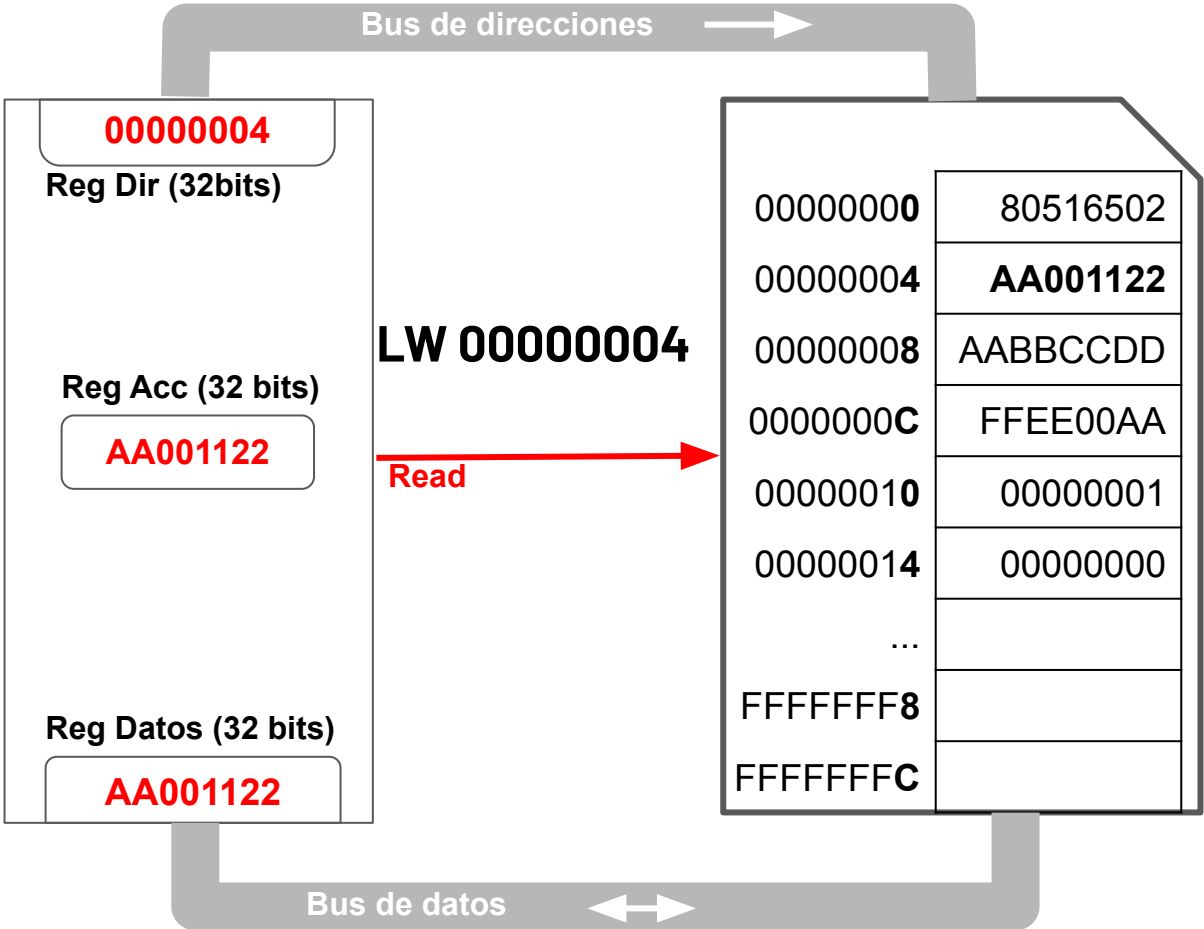
En un acceso alineado se leen 4 bytes en la misma posición de memoria. En este ejemplo se leen 00000008,9,A y B.

# Arquitectura de 32 bits (dir al byte)



En un acceso no alineado a 00000000D , se accede a D,E,F y luego 10 (siguiente palabra). Este tipo de acceso NO es soportado por todas las computadoras!

# Arquitectura de 32 bits (dir al byte)



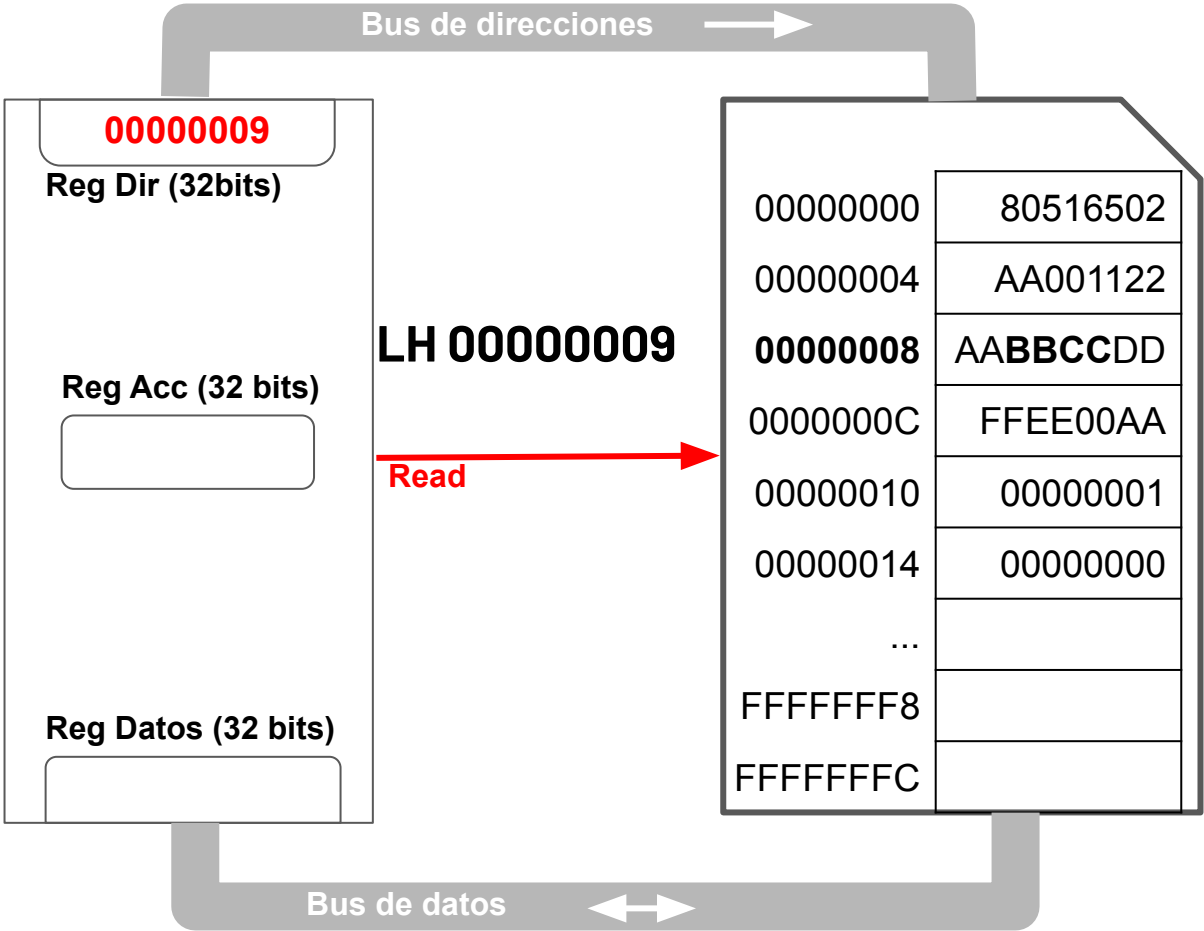
En CPU 32 bits, palabras de memoria de 4 bytes, las direcciones alineadas para LW terminan en

- 0 (00 00)
- 4 (01 00)
- 8 (10 00)
- C (11 00)

Notemos que son múltiplo de 4.

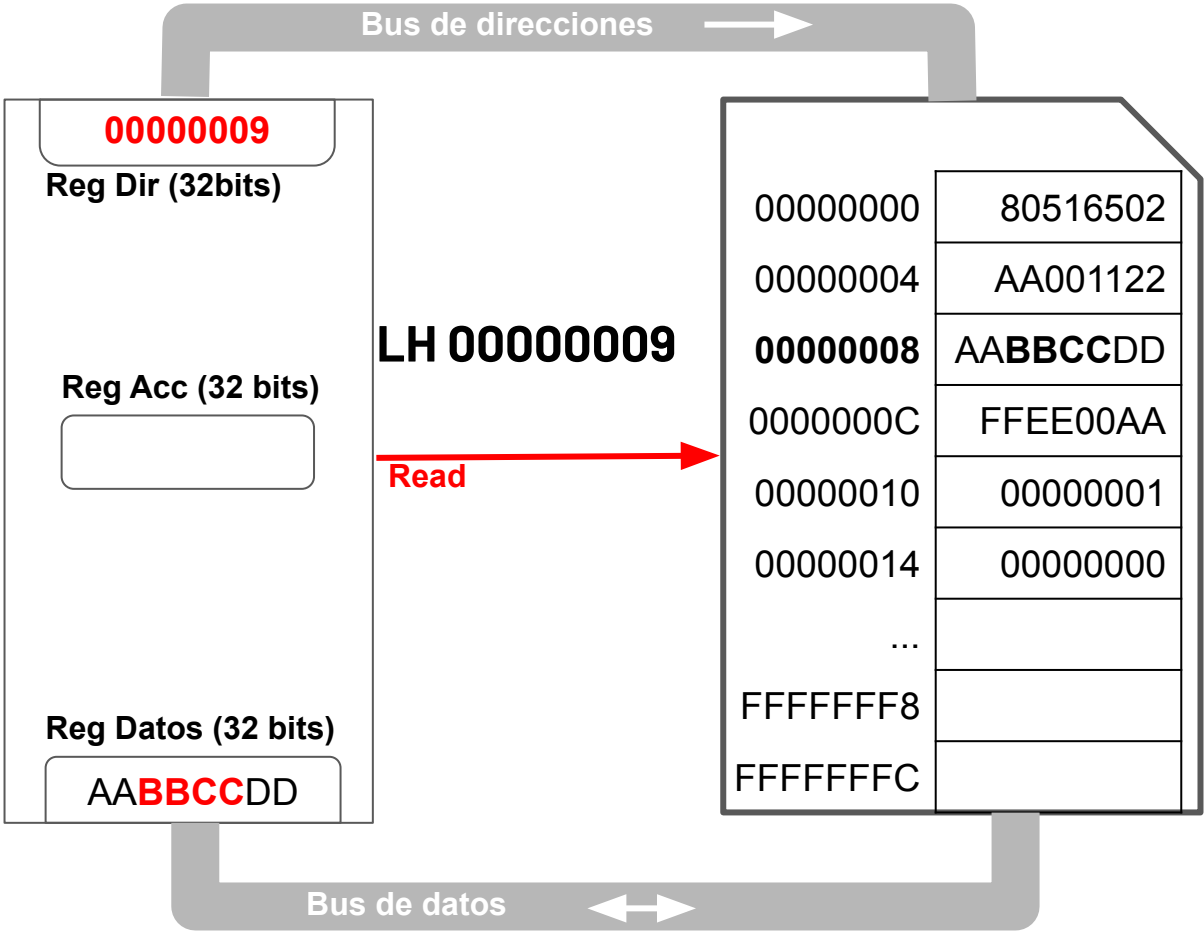


# Arquitectura de 32 bits (dir al byte)



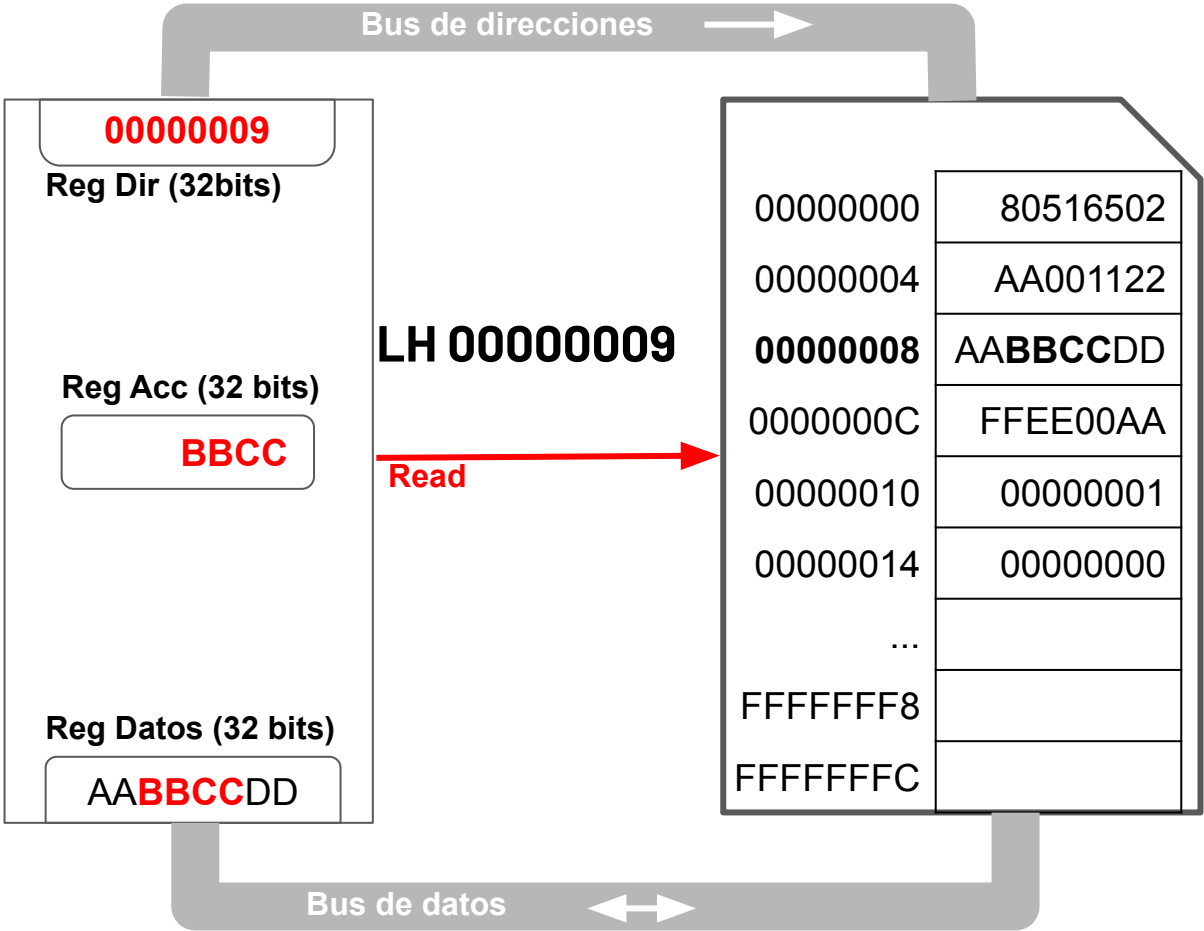
En el caso de LH, leemos 2 bytes de memoria. Con este ejemplo leemos 00000009 y 0000000A , ambos bytes están dentro de 00000008.

# Arquitectura de 32 bits (dir al byte)



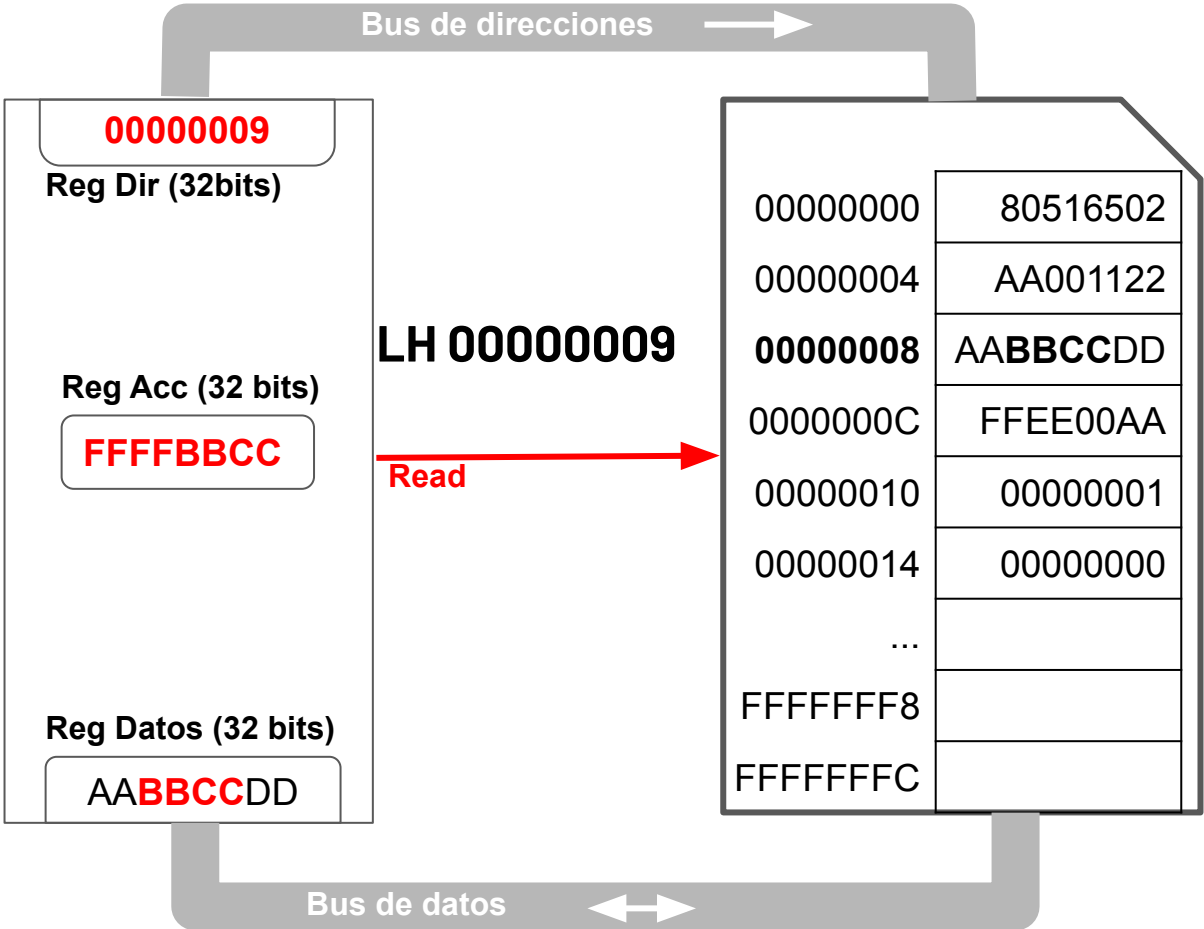
Ahora esos 2 bytes tienen que ajustarse a 32 bits por lo tanto la CPU toma los 2 bytes que vienen de la memoria y los acomoda en la parte baja del acumulador...

# Arquitectura de 32 bits (dir al byte)

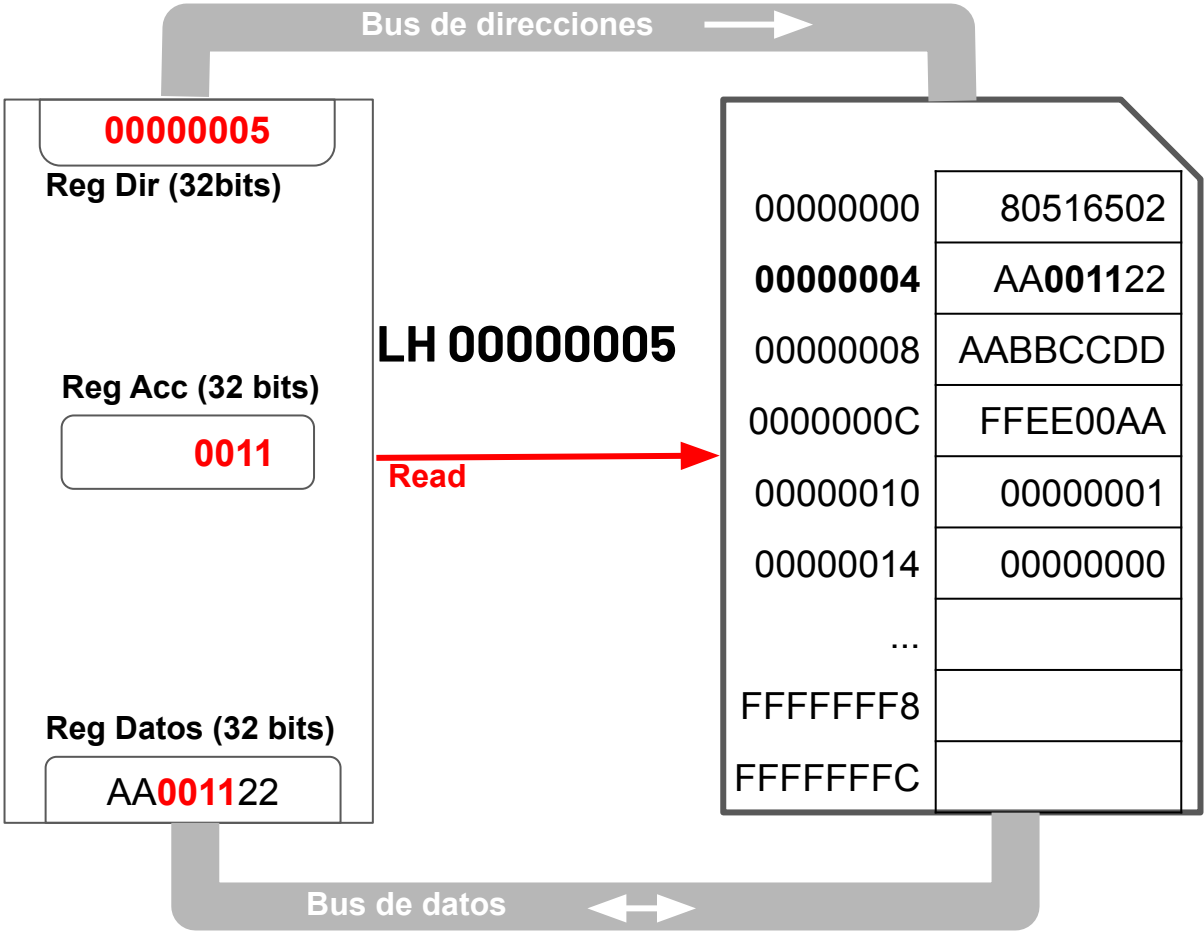


Ahora al ser LH, se propaga el signo, así que B es 1100, bit de signo 1, por ende se completa la parte alta del acumulador con FFFF.

# Arquitectura de 32 bits (dir al byte)

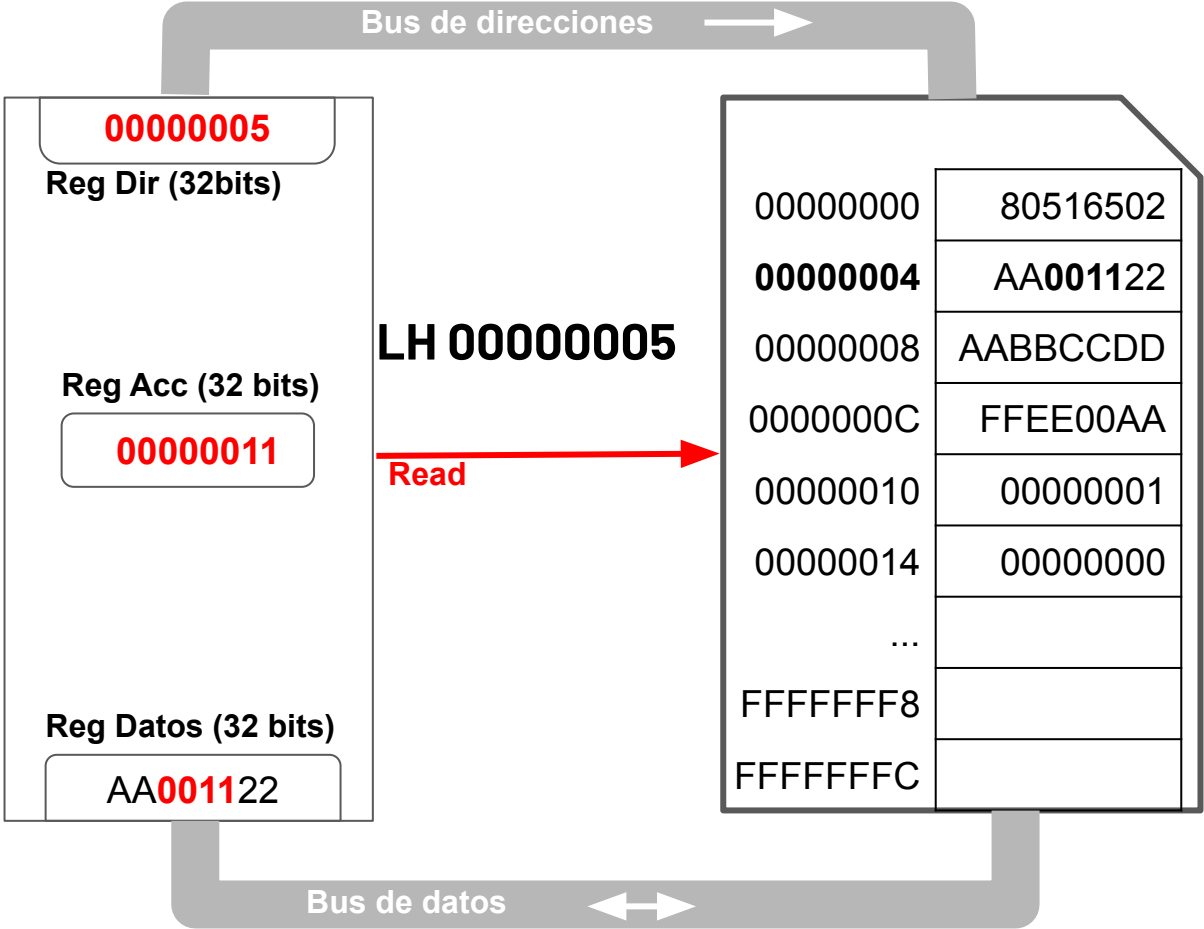


# Arquitectura de 32 bits (dir al byte)

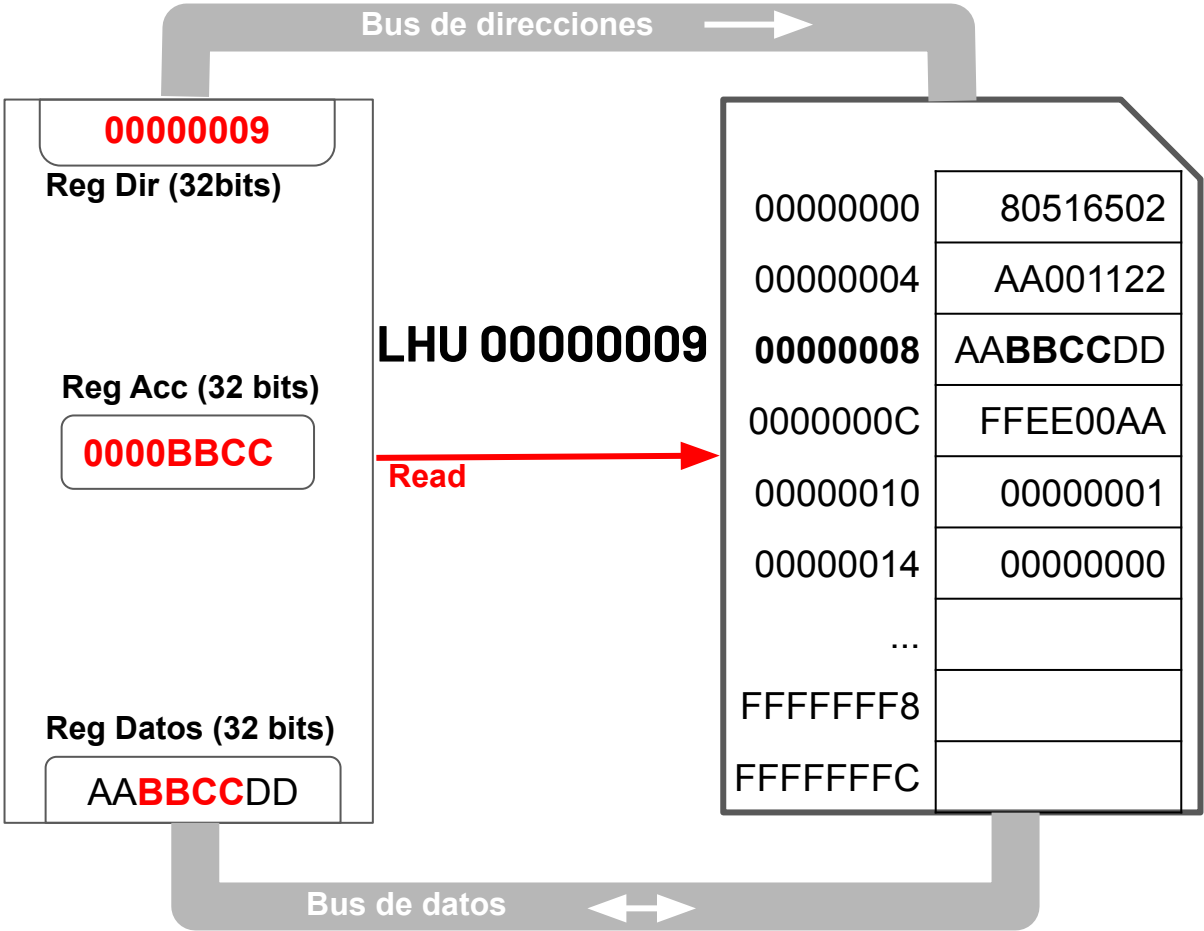


En el caso de haber accedido a 00000005 , notamos que el dato es 0011, se propaga el bit de signo con ceros...

# Arquitectura de 32 bits (dir al byte)

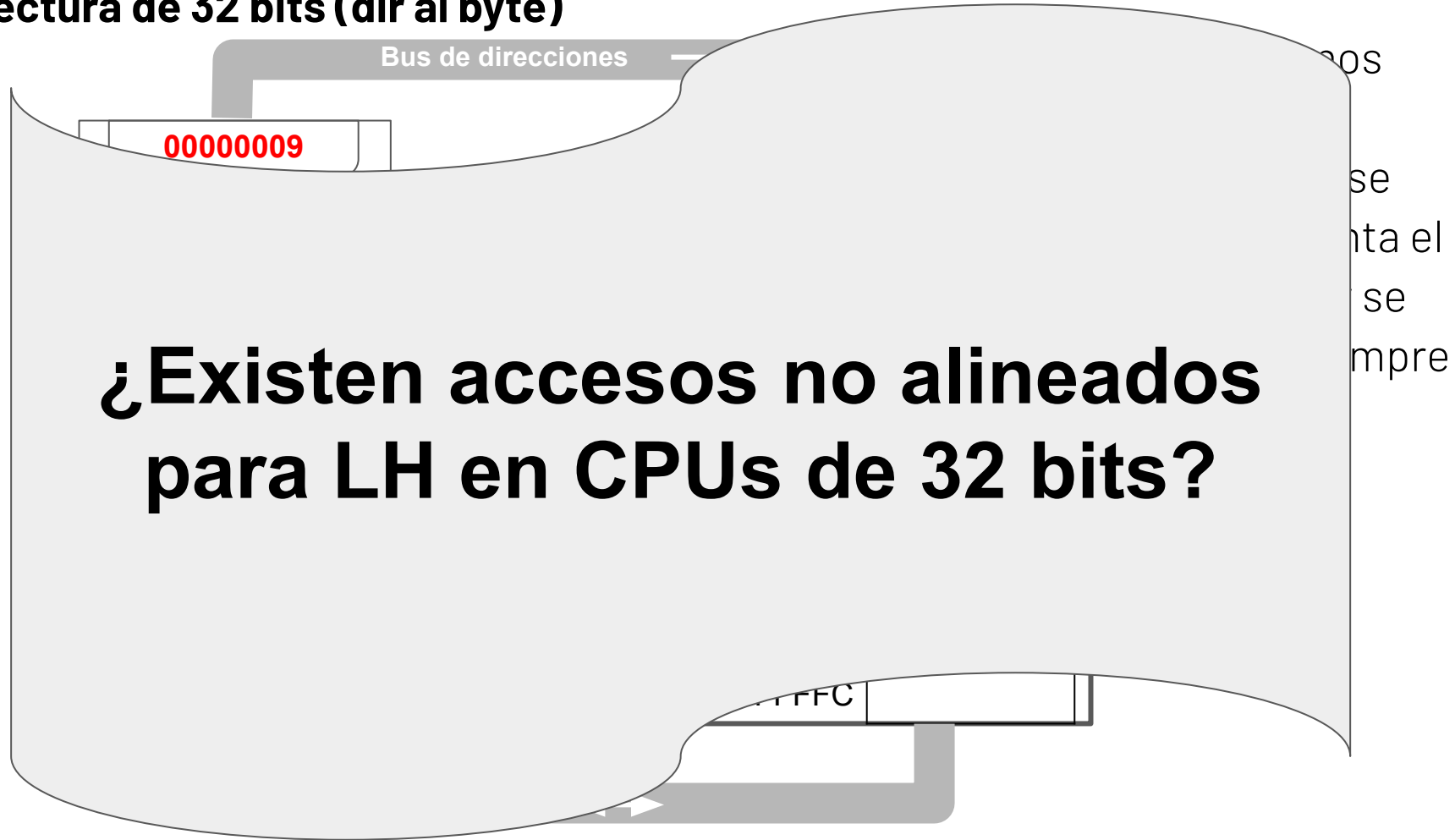


# Arquitectura de 32 bits (dir al byte)



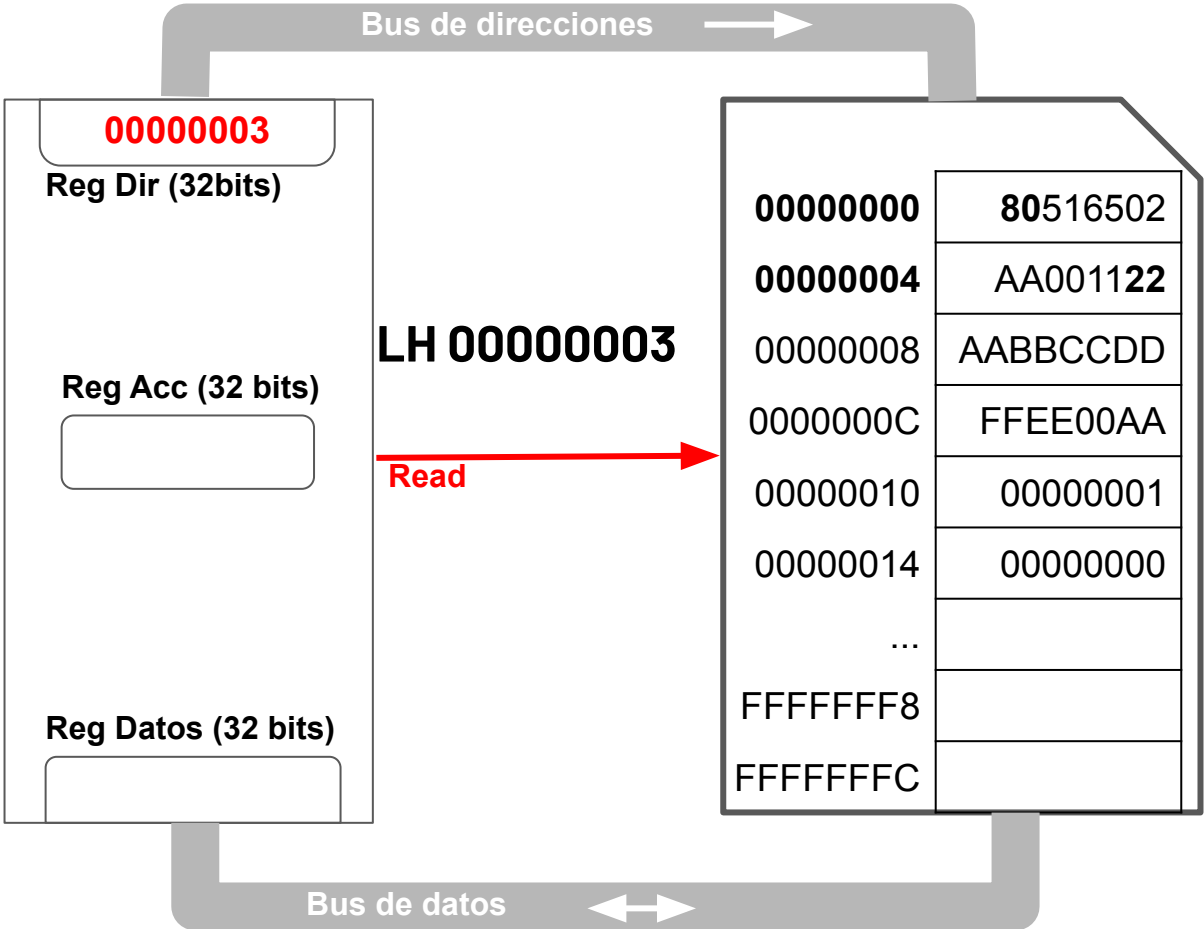
Si hubiésemos usado LHU, entonces no se toma en cuenta el bit de signo y se completa siempre con ceros.

## Arquitectura de 32 bits (dir al byte)





# Arquitectura de 32 bits (dir al byte)



En este ejemplo, LH 00000003, accede a 3 y 4... pero cada byte está en una palabra distinta... por ende NO es alineado.

Esto ocurre siempre que la dir termine en 3 (00 11), 7 (01 11), B (10 11) o F (11 11).

**32 bits**

| Dirección | 3 (11)  | 2 (10)  | 1 (01)  | 0 (00)  |
|-----------|---------|---------|---------|---------|
| 0000 0000 | 0003    | 0002    | 0001    | 0000    |
| 0000 0004 | 0007    | 0006    | 0005    | 0004    |
| 0000 0008 | 000B    | 000A    | 0009    | 0008    |
| 0000 000C | 000F    | 000E    | 000D    | 000C    |
| 0000 0010 | 0013    | 0012    | 0011    | 0010    |
| 0000 0014 | 0017    | 0016    | 0015    | 0014    |
| 0000 0018 | 001B    | 001A    | 0019    | 0018    |
| 0000 001C | 001F    | 001E    | 001D    | 001C    |
| 0000 0020 | 0023    | 0022    | 0021    | 0020    |
| 0000 0024 | 0027    | 0026    | 0025    | 0024    |
| ...       |         |         |         |         |
| FFFF FFFC | F..FFFF | F..FFFE | F..FFFD | F..FFFC |

Las palabras (words) son de 4 bytes. Usando LW se accede a 4 bytes consecutivos. Solo las dirs que terminan en 0,4,8 y C están alineadas al word.

Usando LH/LHU se accede a 2 bytes. Solo las dirs que terminan en 3,7,B o F están desalineadas.

Ej: LW 0x00000014 (Alineado)

LW 0x00000015 (No Alineado)

LH 0x00000022 (Alineado)

LH 0x00000023 (No Alineado)

## **Ejemplo:**

Dada una computadora con palabra (word) de 32 bits. Posee las instrucciones LW, LH, LB, LHU y LBU. Utiliza 24 bits para direccionar al byte (000000~FFFFFF). Posee una memoria ROM de 256 Bytes ubicada a partir de la dirección 0. Indique cuál es la primera dirección de esta memoria:

## **Ejemplo:**

Dada una computadora con palabra (word) de 32 bits. Posee las instrucciones LW, LH, LB, LHU y LBU. Utiliza 24 bits para direccionar al byte (000000~FFFFFF).

Posee una memoria ROM de 256 Bytes ubicada a partir de la dirección 0.

Indique cual es la primera dirección de esta memoria: 000000.

Indique cual es la última dirección de esta memoria:

## **Ejemplo:**

Dada una computadora con palabra (word) de 32 bits. Posee las instrucciones LW, LH, LB, LHU y LBU. Utiliza 24 bits para direccionar al byte (000000~FFFFFF).

Posee una memoria ROM de 256 Bytes ubicada a partir de la dirección 0.

Indique cual es la primera dirección de esta memoria: 000000.

Indique cual es la última dirección de esta memoria: 0000FF.

**Ejemplo:**

Dada una computadora con palabra (word) de 32 bits. Posee la instrucciones LW, LH, LB, LHU y LBU. Utiliza 24 bits para direccionar al byte (000000~FFFFFF). Posee una memoria ROM de 256 Bytes ubicada a partir de la dirección 0. Indique cual es la primera dirección de esta memoria: 000000.

Indique cual es la última dirección de esta memoria: 0000FF.

Indique si los siguientes accesos son alineados:

|          |           |           |           |           |           |            |
|----------|-----------|-----------|-----------|-----------|-----------|------------|
| Instr.   | LW 000010 | LW 000011 | LH 000012 | LH 000013 | LB 000013 | LHU 000001 |
| Alineado |           |           |           |           |           |            |

**Ejemplo:**

Dada una computadora con palabra (word) de 32 bits. Posee la instrucciones LW, LH, LB, LHU y LBU. Utiliza 24 bits para direccionar al byte (000000~FFFFFF). Posee una memoria ROM de 256 Bytes ubicada a partir de la dirección 0. Indique cual es la primera dirección de esta memoria: 000000.

Indique cual es la última dirección de esta memoria: 0000FF.

Indique si los siguientes accesos son alineados:

|          |           |           |           |           |           |            |
|----------|-----------|-----------|-----------|-----------|-----------|------------|
| Instr.   | LW 000010 | LW 000011 | LH 000012 | LH 000013 | LB 000013 | LHU 000001 |
| Alineado | Si        | No        | Si        | No        | Si        | Si         |

# Ejemplo:

Dada una computadora con palabra (word) de 32 bits. Posee la instrucciones LW, LH, LB, LHU y LBU. Utiliza 24 bits para direccionar al byte (000000~FFFFFF). Posee una memoria ROM de 256 Bytes ubicada a partir de la dirección 0. Si la memoria se encuentra cargada de la siguiente forma, indique el dato...

|        | 0          | 4          | 8          | C          |
|--------|------------|------------|------------|------------|
| 00000_ | 0x00112233 | 0x44556677 | 0x8899AABB | 0xCCDDEEFF |
| 00001_ | 0x80516502 | 0x80864004 | 0xFFEE0000 | 0x01234567 |
| 00002_ | 0x55AAAA55 | 0x80808181 | 0x30313233 | 0x00000000 |

**LW 000004 =**



# Ejemplo:

Dada una computadora con palabra (word) de 32 bits. Posee la instrucciones LW, LH, LB, LHU y LBU. Utiliza 24 bits para direccionar al byte (000000~FFFFFF). Posee una memoria ROM de 256 Bytes ubicada a partir de la dirección 0. Si la memoria se encuentra cargada de la siguiente forma, indique el dato...

|        | 0          | 4          | 8          | C          |
|--------|------------|------------|------------|------------|
| 00000_ | 0x00112233 | 0x44556677 | 0x8899AABB | 0xCCDDEEFF |
| 00001_ | 0x80516502 | 0x80864004 | 0xFFEE0000 | 0x01234567 |
| 00002_ | 0x55AAAA55 | 0x80808181 | 0x30313233 | 0x00000000 |

**LW 000004 = 0x44556677**

# Ejemplo:

Dada una computadora con palabra (word) de 32 bits. Posee la instrucciones LW, LH, LB, LHU y LBU. Utiliza 24 bits para direccionar al byte (000000~FFFFFF). Posee una memoria ROM de 256 Bytes ubicada a partir de la dirección 0. Si la memoria se encuentra cargada de la siguiente forma, indique el dato...

|        | 0          | 4          | 8          | C          |
|--------|------------|------------|------------|------------|
| 00000_ | 0x00112233 | 0x44556677 | 0x8899AABB | 0xCCDDEEFF |
| 00001_ | 0x80516502 | 0x80864004 | 0xFFEE0000 | 0x01234567 |
| 00002_ | 0x55AAAA55 | 0x80808181 | 0x30313233 | 0x00000000 |

**LW 000005 = 0xBB445566 (desalineado!!!)**

# Ejemplo:

Dada una computadora con palabra (word) de 32 bits. Posee la instrucciones LW, LH, LB, LHU y LBU. Utiliza 24 bits para direccionar al byte (000000~FFFFFF). Posee una memoria ROM de 256 Bytes ubicada a partir de la dirección 0. Si la memoria se encuentra cargada de la siguiente forma, indique el dato...

|        | 0          | 4          | 8          | C          |
|--------|------------|------------|------------|------------|
| 00000_ | 0x00112233 | 0x44556677 | 0x8899AABB | 0xCCDDEEFF |
| 00001_ | 0x80516502 | 0x80864004 | 0xFFEE0000 | 0x01234567 |
| 00002_ | 0x55AAAA55 | 0x80808181 | 0x30313233 | 0x00000000 |

**LH 000005 = 0x00005566**

**LHU 000005 = 0x00005566**

**LH 00000A = 0xFFFF8899**

**LHU 00000A = 0x00008899**

**Ejemplo:**

Dada una computadora con palabra (word) de 32 bits. Posee la instrucciones LW, LH, LB, LHU y LBU. Utiliza 24 bits para direccionar al byte (000000~FFFFFF). Posee una memoria ROM de 256 Bytes ubicada a partir de la dirección 0. Si la memoria se encuentra cargada de la siguiente forma, indique el dato...

|        | 0          | 4          | 8          | C          |
|--------|------------|------------|------------|------------|
| 00000_ | 0x00112233 | 0x44556677 | 0x8899AABB | 0xCCDDEEFF |
| 00001_ | 0x80516502 | 0x80864004 | 0xFFEE0000 | 0x01234567 |
| 00002_ | 0x55AAAA55 | 0x80808181 | 0x30313233 | 0x00000000 |

**LB 000008 = FFFFFFFB**

**LBU 000008 = 000000BB**

# Byte Ordering

Si la información que debemos manejar se representa con más de un byte, tenemos distintas formas de ordenar cada byte en memoria.

Si tenemos un dato de 2 bytes, ej: 0x6502, y vamos a guardar ese dato a partir de la posición de memoria 0000 (dir al byte) en una computadora de 16 bits, podemos guardarlo como:

| Big Endian (BE) |         |        |
|-----------------|---------|--------|
| Dirección       | High(1) | Low(0) |
| 0000            | 02      | 65     |

**LB 0000 = 0x65**

**LB 0001 = 0x02**

| Little Endian (LE) |         |        |
|--------------------|---------|--------|
| Dirección          | High(1) | Low(0) |
| 0000               | 65      | 02     |

**LB 0000 = 0x02**

**LB 0001 = 0x65**

Big-Endian significa que en la dir Big (0001 es mayor que 0000) se encuentra el endian (parte baja el número). El caso opuesto es Little-Endian donde en la dir Little (0000 menor a 0001) se encuentra el endian.

# Byte Ordering

En el caso de palabras de 4 bytes, también tenemos el mismo esquema. El dato 0x26062011, tiene su byte más significativo como 0x26 y el menos significativo como 0x11.

| Big Endian (BE) |       |       |       |       |
|-----------------|-------|-------|-------|-------|
| Dirección       | 3(11) | 2(10) | 1(01) | 0(00) |
| 0000            | 11    | 20    | 06    | 26    |

**LB 0000 = 0x26**

**LB 0001 = 0x06**

**LB 0002 = 0x20**

**LB 0003 = 0x11**

| Little Endian (LE) |       |       |       |       |
|--------------------|-------|-------|-------|-------|
| Dirección          | 3(11) | 2(10) | 1(01) | 0(00) |
| 0000               | 26    | 06    | 20    | 11    |

**LB 0000 = 0x11**

**LB 0001 = 0x20**

**LB 0002 = 0x06**

**LB 0003 = 0x26**

# Ejemplo:

Dada una computadora con palabra (word) de 32 bits. Posee la instrucción LW, indique para cada tipo de byte ordering como se lee el dato...

|        | 0          | 4          | 8          | C          |
|--------|------------|------------|------------|------------|
| 00000_ | 0x00112233 | 0x44556677 | 0x8899AABB | 0xCCDDEEFF |
| 00001_ | 0x80516502 | 0x80864004 | 0xFFEE0000 | 0x01234567 |
| 00002_ | 0x55AAAA55 | 0x80808181 | 0x30313233 | 0x00000000 |

**LW 000008 = BBAA9988 (Big-Endian)    LW 000008 = 8899AABB (Little-Endian)**